

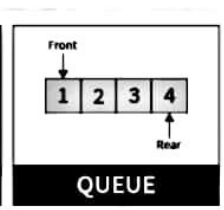
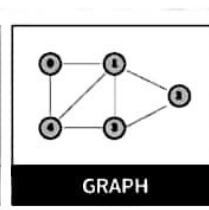
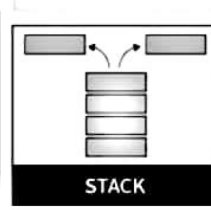
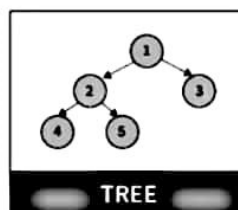
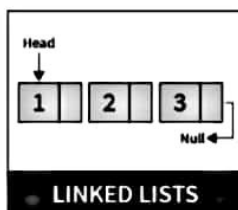
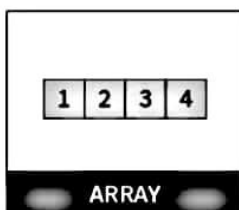


DATA STRUCTURE

What is Data Structure:

A data structure is a storage that is used to store and Organize data. It is a way of arranging data on a computer so that it can be accessed and update efficiently.

A data structure is not only used for organizing the data. It is also used for processing, retrieving, and storing data.



HOW DATA STRUCTURE VARIES FROM DATA TYPE

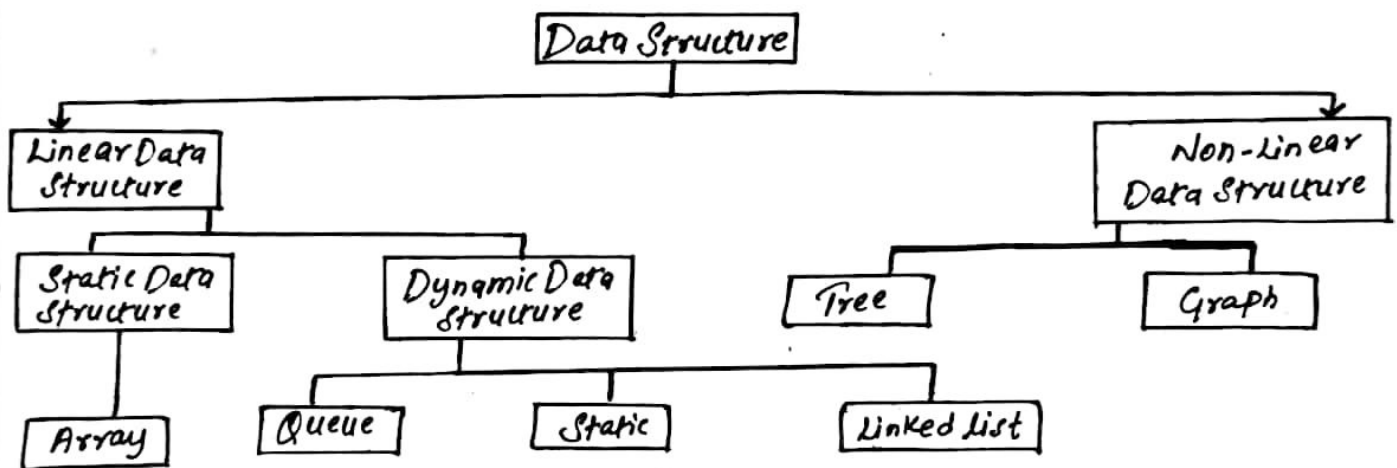
We already have learned about data structure. Many times, what happens is that people get confused between data type and data structure. So let's see a few differences between data type and data structure to make it clear.

<u>DATA TYPE</u>	<u>DATA STRUCTURE</u>
➤ The data type is the form of a variable to which a value can be assigned. It defines that the particular variable will assign the value of the given data type only.	➤ Data structure is a collection of different kinds of data. That entire data can be represented using an object and can be used throughout the program.
➤ It can hold value but not data. Therefore, it is dataless.	➤ It can hold multiple of data within a single object.
➤ The implementation of a data type is known as abstract implementation.	➤ Data structure implementation is known as concrete implementation.
➤ There is no time complexity in the case of data type.	➤ In data structure objects, time complexity plays an important role.
➤ In the case of data types, the value of data is not stored because it only represents the data type that can be stored.	➤ While in the case of data structures, the data and its value acquire the space in the computer's main memory. Also, a data structure can hold different kinds and types of data within one single object.
➤ Data type examples are int, float, double, etc.	➤ Data structure examples are stack, queue, tree, etc.

CLASSIFICATION OF DATA STRUCTURE

Data structure has many different uses in our daily life. There are many different data structures that are used to solve different mathematical and logical problems. By using data structure, one can organize and process a very large amount of data in a relatively short period. Let's look at different data structures that are used in different situations.

Classification of data structure



Linear data structure: Data structure in which data elements are arranged sequentially or linearly, where each element is attached to its previous and next adjacent elements, is called a linear data structure.

Examples:- array, stack, queue, linked list, etc.

- **Static data structure:** Static data structure has a fixed memory size. It is easier to access the elements in a static data structure.
Examples:- Array.

DATA STRUCTURE

Data : Anything to give information is called data.

Ex:- Student Name, Student Roll No.

Structure: Representation of data is called structure.

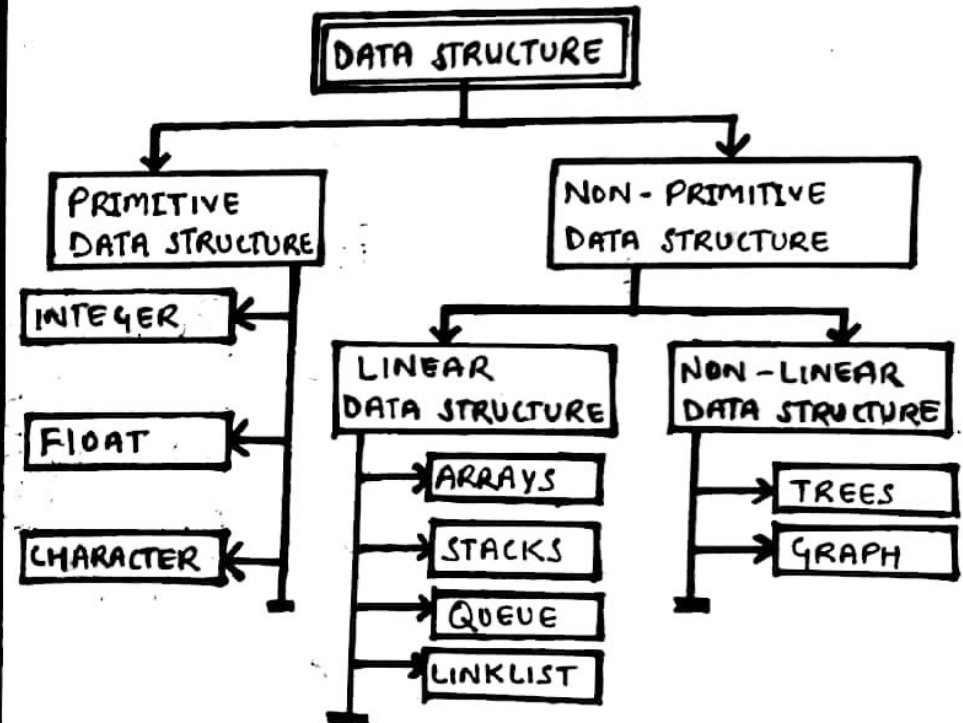
Ex:- Graph, Arrays, List.

Data structure:

- Data structure = Data + structure.
- Data structure is a way to store and Organize data so that it can be used efficiently (better way).
- Data structure is a way of organizing all data items and relationship to each other.

Types of Data structure :

There are mainly two types of data structure.



Primitive data structure: These are basic structure and are directly operated by machine instruction.
Ex:- Integer, float, character.

- **Dynamic data structure:-** In dynamic data structure, the size is not fixed. It can be randomly updated during the runtime which may be considered efficient concerning the memory (Space) complexity of the code.

Examples:- Queue, stacks, etc.

Non-Linear data structure: Data structures where data elements are not placed sequentially or linearly are called non-linear data structures.

In a non-linear data structure, we can't traverse all the elements in a single run only.

Examples of Non-linear data structures are :- trees and graph.

NEED OF DATA STRUCTURE :-

The structure of the data and the synthesis of the algorithm are relative to each other.

Data Presentation must be easy to understand so the developer, as well as the user, can make an efficient implementation of the operation.

Data structures provide an easy way of organizing, retrieving, managing, and storing data.

Here is a list of the needs for data.

1. Data structure modification is easy.
2. It requires less time.
3. Save storage memory space.
4. Data representation is easy.
5. Easy access to the large database.

ARRAYS:

An array is a linear data structure and it is a collection of items stored at contiguous memory locations. The idea is to store multiple items of the same type together in one place. It allows the processing of a large amount of data in a relatively short period. The first element of the array is indexed by a subscript of 0. There are different operations possible in an array, like Searching, Sorting, Inserting, Traversing, Reversing, and Deleting.

Memory Location									
200	201	202	203	204	205	206	■	■	■
U	B	F	D	A	E	C	■	■	■
0	1	2	3	4	5	6	■	■	■
Index.									

Characteristics of an Array:

An array has various characteristics which are as follows:

- ➔ Arrays use an index-based data structure which helps to identify each of the elements in an array easily using the index.
- ➔ If a user wants to store multiple values of the same data type, then the array can be utilized efficiently.
- ➔ An array can also handle complex data structures by storing data in a two-dimensional array.
- ➔ An array is also used to implement other data structures like Stacks, Queues, Heaps, Hash tables, etc.
- ➔ The search process in an array can be done very easily.

Applications of Array :

Different applications of an array are as follows:

- An array is used in solving matrix problems.
- Database records are also implemented by an array.
- It helps in implementing a sorting algorithm.
- It is also used to implement other data structures like stacks, queues, heaps, hash tables, etc.
- An array can be used for CPU scheduling.
- Can be applied as a lookup table in computers.
- Array can be used in speech processing where every speech single is an array.
- The screen of the computer is also displayed by an array. Here we use a multidimensional array.
- The array is used in many management system like a library, students, parliament, etc.
- The array is used in the online ticket booking system. Contacts on a cell phone are displayed by this array.
- In games like online chess, where the player can store his past moves as well as current moves. It indicates a hint of position.
- To save images in a specific dimension in the android like 360×1200

Real-Life Application Array :

- An array is frequently used to store data for mathematical computations.
- It is used in image processing.
- It is also used in record management.
- Book pages are also real-life examples of an array.
- It is used in ordering boxes as well.

Non-Primitive data structure:

These are derived from the primitive data structure. its a collection of same type or different type primitive data structure.

Ex:- Arrays, stack, trees.

Data Structure Operation

The data which is stored in our data structure are processed by some set of operation.

- 1) **Inserting:** Add a new data in the data structure.
- 2) **Deleting:** Remove a data from the data structure.
- 3) **Sorting:** Arrange data increasing or decreasing order.
- 4) **Searching:** Find the location of data in data structure.

5) **Merging:** Combining the data of two different stored files into a single stored file.

6) **Traversing:** Accessing each data exactly one in the data structure so that each data items is traversed or visited.

Arrays

- An Array can be defined as an infinite collection of homogeneous (similar type) elements.
- Array are always stored in consecutive (specific) memory location.
- Array can be stored multiple values which can be referenced by a single name.

TYPES OF ARRAYS

SINGLE DIMENTION ARRAY

MULTI DIMENTION ARRAY

1). Single Dimensional Arrays :

It is also known as One Dimensional (1D) Array.

- It's use only one subscript to define the elements of Arrays.

[row] [column]

Declaration :

data-type var_name [expression];
Ex:- int num [10];
char c [5];

size

Initializing one - Dimensional Array :

Data-type var_name [expression] = {values};

Ex:- int num [10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
char a [5] = {'A', 'B', 'C', 'D', 'E'};

2). Multi-Dimensional Arrays :

Multi-Dimensional Arrays use more than one subscript to describe the Arrays elements.

[] [] [] ---

Two Dimensional Arrays :

It's use two [row] [column] subscript, one subscript to represent row value and second subscript to represent column value.

It mainly use for matrix Representation.

Declaration two Dimensional Arrays :

Data-type var-name [rows] [column]

Ex:- int num [3] [2]

Initialization 2-D Arrays :

data-type var-name [rows] [column] = {values};

Ex:- int num [3] [2] = {1, 2, 3, 4, 5, 6};

or
int num [] [] = {1, 2, 3, 4, 5, 6};

$\begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}_{3 \times 2}$

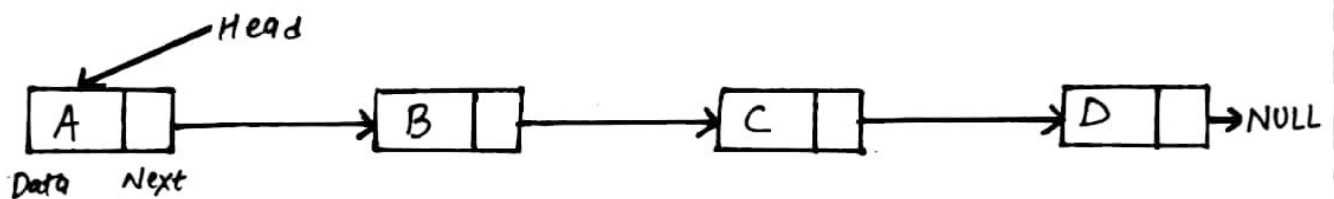
num [0] [0] = 1
num [0] [1] = 2
num [1] [0] = 3
num [1] [1] = 4
num [2] [0] = 5
num [2] [1] = 6

LINKED LIST

A linked-list is a linear data structure in which elements are not stored at contiguous memory locations. The elements in a linked list are linked using pointers as shown in the below image:

Types of linked list:

- Singly-linked list.
- Doubly-linked list.
- Circular linked list.
- Doubly circular linked list.



Characteristics of a Linked List:

A linked list has various characteristics which are as follows:

- A linked list uses extra memory to store links.
- During initialization of linked list, there is no need to know the size of the element.
- Linked lists are used to implement stacks, queues, graphs, etc.
- The first node of the linked list is called the Head.
- The next pointer of the last node always points to NULL.
- In linked list, insertion and deletion is possible easily.
- Each node of the linked list consists of a pointer/link which is the address of the next node.
- Linked list can shrink or grow at any point in time easily.

Applications of the Linked List:

Different applications of linked list are as follows:

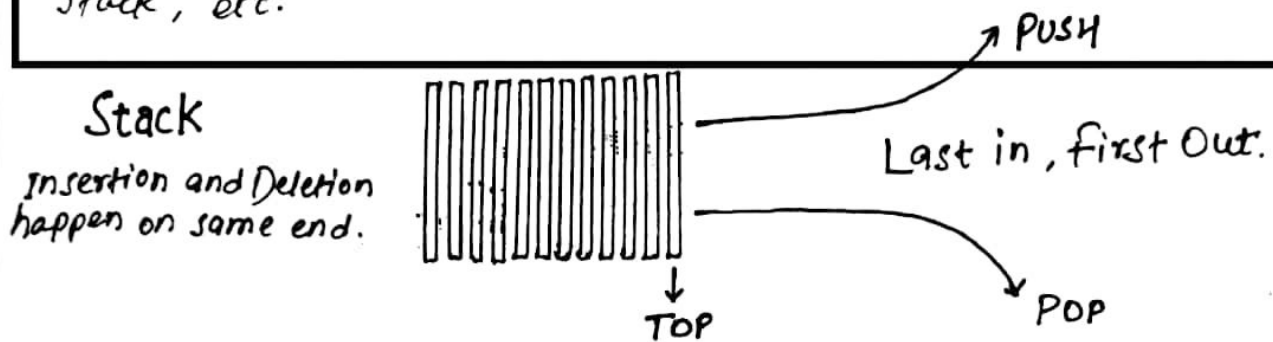
- Linked list are used to implement stacks, queues, graphs, etc.
- Linked list are used to perform arithmetic operations on long integers.
- It is used for the representation of sparse matrices.
- It is used in linked allocation of files.
- It helps in memory management.
- It is used in the representation of polynomial Manipulation where each polynomial term represents a node in the linked list.
- Linked lists are used to display image containers. User can visit past, current, and next images.
- They are used to store the history of the visited page.
- They are used to perform undo operations.
- Linked are used in software development where they indicate the correct syntax of a tag.
- Linked lists are used to display social media feeds.

Real-Life Applications of a Linked List:

- A linked list is used in Round-Robin scheduling to keep track of the turn in multi-player games.
- It is used in Image viewer. The previous and next images are linked, hence can be accessed by the previous and next buttons.
- In a music playlist, songs are linked to the previous and next songs.

Stack

Stack is a linear data structure that follows a particular order in which the operations are performed. The order is LIFO (Last In First Out). Entering and retrieving data is possible from only one end. The entering and retrieving of data is also called push and pop operation in a stack. There are different operations possible in a stack like reversing a stack using recursion, sorting, Deleting the middle element of a stack, etc.



Characteristics of a stack:

Stack has various different characteristics which are as follows:

- Stack is used in many different algorithms like Tower of Hanoi, tree, traversal, recursion etc.
- Stack is implemented through array or linked list.
- It follows Last In First Out operation i.e., element which is inserted first will pop in last and vice versa.
- The insertion and deletion happens at one end i.e. from the top of the stack.
- In stack, if allocated space for stack is full, and still anyone attempts to add more elements, it will lead to stack overflow.

Applications of Stack:

Different applications of stacks are as follows:

- Stack data structure is used in evaluation and conversion of arithmetic expressions.
- Stack is used in Recursion.
- It is used for parenthesis checking.
- While reversing a string, stack is used as well.
- Stack is used in memory management.
- It is also used for processing of function calls.
- The stack is used to convert expressions from infix to postfix.
- The stack is used to perform undo as well as redo operations in word processors.
- The stack is used in virtual machines like JVM.
- The stack is used in the media players. Useful to play the next and previous song.
- The stack is used in recursion operation.

Real Life Application of Stack:

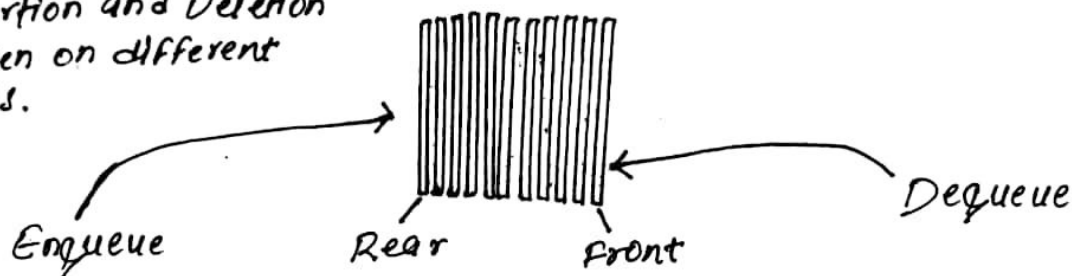
- Real life example of a stack is the layer of eating plates arranged one above the other. When you remove a plate from the pile, you can take the plate on the top of the pile. But this is exactly the plate that was added most recently to the pile. If you want the plate at the bottom of the pile, you must remove all the plates on the top of it to reach it.
- Browsers use stack data structure to keep track of previously visited sites.
- Call log in mobile also uses stack data structure.

QUEUE

Queue is a linear data structure that follows a particular order in which the operations are performed. The order is First In First Out (FIFO) i.e. the data item stored first will be accessed first. In this, entering and retrieving data is not done from only one end. An example of a queue is any queue of consumers for a resource where the consumer that came first is served first. Different operations are performed on queue like Reversing a Queue (with or without using recursion), Reversing the first K elements of a Queue, etc. Few basic operations performed in queue are enqueue, dequeue, front, rear, etc.

Queue

Insertion and Deletion happen on different ends.



Characteristics of a Queue:

Queue has various different characteristics which are as follow:

- > Queue is a FIFO (First In First Out) structure.
- > To remove the last element of Queue, all the elements inserted before the new element in the queue must be removed.
- > A queue is an ordered list of elements of similar data types.

Applications of Queue:

Different applications of Queue are as follows:

- Queue is used for handling website traffic.
- It helps to maintain the playlist in media players.
- Queue is used in Operating systems for handling interrupts.
- It helps in serving requests on a single shared resource, like a printer, CPU task scheduling, etc.
- It is used in asynchronous transfer of data for eg. pipes, file IO, sockets.
- Queue are used for job scheduling in operating system.
- In social media to upload multiple photos or videos queue is used.
- To send an e-mail queue data structure is used.
- To handle website traffic at a time queue are used.
- In window operating system, to switch multiple application.

Real-Life Applications of Queue

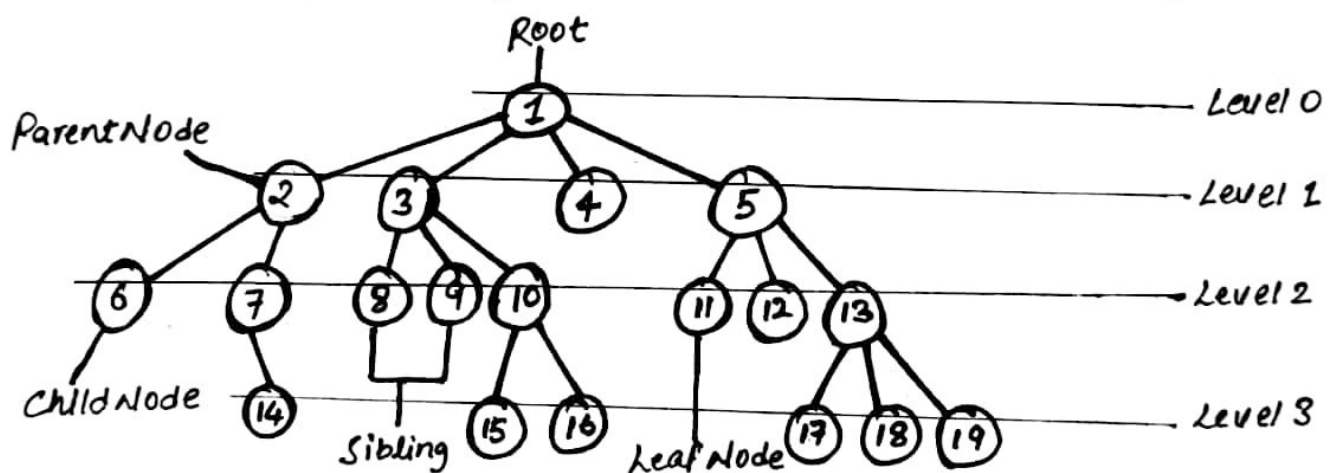
- A real world example of Queue is a single lane one way road, where the vehicle that enters first will exit first.
- A more real-world example can be seen in the queue at the ticket windows.
- Cashier line in a store is also an example of queue.
- People on an escalator.

TREE

A tree is a non-linear and hierarchical data structure where the elements are arranged in a tree-like structure. In a tree, the topmost node is called the root node. Each node contains some data, and data can be of any type. It consists of a central node, structural nodes, and sub-nodes which are connected via edges. Different tree data structures allow quicker and easier access to the data as it is a non-linear data structure. A tree has various terminologies like Node, Root, Edge, Height of tree, Degree of a tree, etc.

There are different types of Tree like.

- Binary Tree , • Binary search tree , • AVL Tree , • B-Tree, etc.



Characteristics of a Tree:

- > A tree is also known as Recursive data structure.
- > In a tree, height of the root can be defined as the longest path from the root node to the leaf node.
- > In a tree, one can also calculate the depth from the top of any node. The root node has depth of 0.

Application of Tree :

Different applications of Tree are as follows:

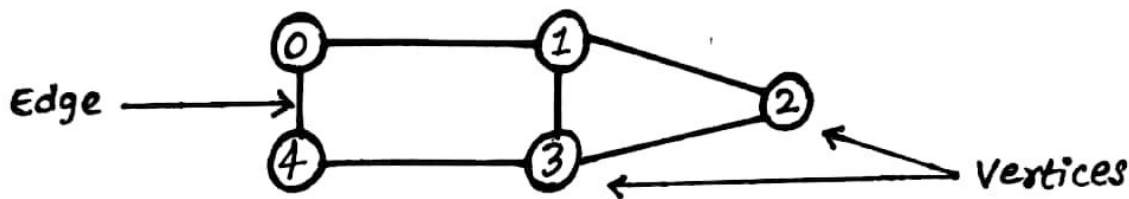
- Heap is a tree data structure that is implemented using arrays, and used to implement priority queues.
- B-Tree and B+ Tree are used to implement indexing in database.
- Syntax Tree helps in scanning, parsing, generation of code and evaluation of arithmetic expressions in compiler design.
- K-D Tree is a space partitioning tree used to organize points in K-dimensional space.
- Spanning trees are used in routers in computer networks.

Real Life Applications of Tree :

- In real life, tree data structure helps in Game Development.
- It also helps in indexing in database.
- Decision Tree is an efficient machine learning tool, commonly used in decision analysis.
It has a flowchart-like structure that helps to understand data.
- Domain Name Server also uses tree data structure.
- The most common use case of a tree is any social networking site.

GRAPH

A graph is a non-linear data structure that consists of vertices (or nodes) and edges. It consists of a finite set of vertices and set of edges that connect a pair of nodes. Graph is used to solve the most challenging and complex programming problems. It has different terminologies which are path, Degree, Adjacent vertices, connected components, etc.



Characteristics of Graph:

Graph has various different characteristics which are as follow:

- The maximum distance from a vertex to all the other vertices is considered as the Eccentricity of that vertex.
- The vertex having minimum Eccentricity is considered the central point of the graph.
- The minimum value of Eccentricity from all vertices is considered as the radius of a connected graph.

Application of Graph:

Different applications of Graph are as follows:

- Graph is used to represent the flow of computation.
- It is used in modeling graph.
- The operating system uses Resource Allocation Graph.
- Also used in the World Wide Web where the web pages represent the nodes.

Real-Life Applications of Graph :

- One of the most common real-world examples of a graph is Google Maps where cities are located as vertices and path connecting those vertices are located as edges of the graph.
- A social network is also one real-world example of a graph where every person on the network is a node, and all of their friendships on the network are the edges of the graph.
- A graph is also used to study molecules in physics and chemistry.

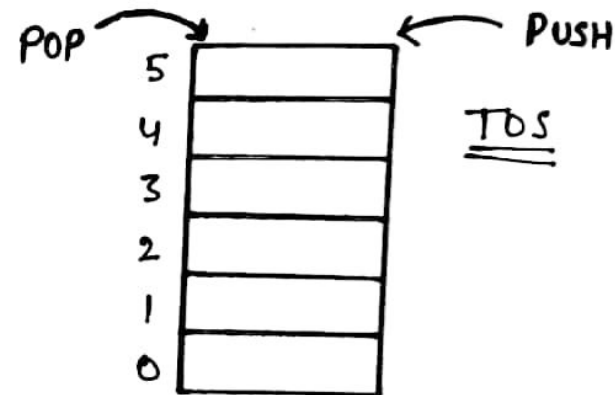
Write a program to read & write one Dimensional Array.

include <stdio.h> → Standard input out
Print f Scan f
include <conio.h> → console input out
clrscr (rc), getch ()

```
void main ( )  
{  
    int a [10], i;  
    clrscr (rc);  
    printf ("Enter the Array Elements");  
    for (i=0; , i<=9; , i++)  
    {  
        scanf ("%d", &a[i]);  
    }  
    printf ("the entered Array is");  
    for (i=0; i<=9; i++)  
    {  
        printf ("%d\n", a[i]);  
    }  
    getch ( );  
}
```

Stacks (Data Structure)

- Stack is a Non-Primitive Linear data structure.
- It is an ordered list in which addition of new data item and deletion of already existing data item is done from only one end known as Top of stack (TOS)



- The last added element will be the first to be Removed from the stack.
This is the reason stack is called Last-in-first-out (LIFO) type of list.

Operations on stack

There are two operation of stack.

1). **Push Operation:** • The process of adding a new element to the top of stack is called Push Operation.

- Every new element is adding to stack top is incremented by one
- In case the array is full and no new element can be added it's called Stack full or Stack overflow condition.

2). **POP Operation:** • The process of deleting an element from the top of stack called POP Operation

- After every POP operation the stack (TOP) is decremented by One.
- If there is no element on the stack and the POP is performed then this will result into Stack underflow condition.

Stack Operation & Algorithm

Stack has two Operation.

- 1). Push operation.
- 2). POP Operation.

1). **PUSH Operation:** • The process of adding a new element of the top of stack is called Push Operation

- Every Push operation Top is incremented by one.

$$\text{TOP} = \text{TOP} + 1$$

- In case the Array is full no new element is added. this condition is called Stack full or Stack Overflow condition.

Algorithm for Inserting an item into the stack (Push operation).

PUSH (stack [max size], item)

Step 1: initialize

set $top = -1$

Step 2: Repeat steps 3 to 5 until $Top < maxsize - 1$

Step 3: Read Item.

Step 4: Set $top = top + 1$

Step 5: Set $stack[Top] = item$

Step 6: Print "Stack overflow"

Algorithm for deleting an item from the stack (POP)

POP (stack [max size], item)

Step 1: Repeat steps 2 to 4 until $Top \geq 0$.

Step 2: Set $item = stack[Top]$.

Step 3: Set $top = top - 1$

Step 4: Print, No. deleted is, Item

Step 5: Print stack under flows.

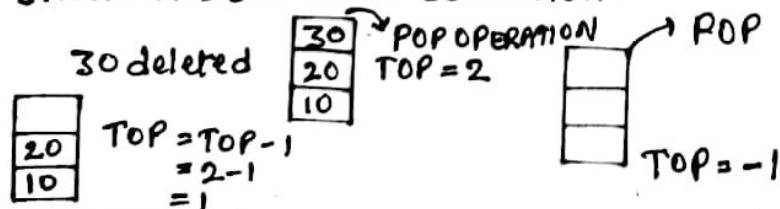
2. POP Operation:

- The process of deleting an element from the top of stack is called POP Operation.

- After every POP Operation the stack TOP is decremented by one.

$Top = Top - 1$

- If there is no element on the stack and the POP operation is performed then this will result into **STACK UNDERFLOW** condition.



Stacks (Prefix & Postfix)

1) Infix Notation: Where the operator is written in between the operands.

Ex:- $A + B$ + operator A, B operands.

2) Prefix Notation: In this operator is written before the operands.

It is also known as Polish Notation.

Ex:- $+AB$

3) Postfix Notation: In this operator is written after the operands.

It is also known as Suffix Notation.

Ex:- $AB +$

Q $\Rightarrow$ Convert the following Infix to Prefix and postfix for $(A+B) * C/D + E^{\wedge}F/G$

Prefix $\Rightarrow (A+B) * C/D + E^{\wedge}F/G$

$+ AB * C/D + E^{\wedge}F/G$

Let $+ AB = R_1$

$R_1 * C/D + E^{\wedge}F/G$

$R_1 * C/D + ^{\wedge}EF/G$

Let $\wedge EF = R_2$

$R_1 * C/D + R_2/G$

$R_1 * /CD + R_2/G$

Let $\Rightarrow /CD = R_3$

$R_1 * R_3 + R_2/G$

$R_1 * R_3 + /R_2G$

Let $\Rightarrow /R_2G = R_4$

$R_1 * R_3 + R_4$

$* R_1 R_3 + R_4$

Let $* R_1 R_3 = R_5$

$R_5 + R_4$

$+ R_5 R_4$

Now enter the value of R_5, R_4, R_3, R_2, R_1

$+ * R_1 R_3 / R_2 G$

$+ * + AB / CD / ^{\wedge}EF G$

13

Postfix $\Rightarrow (A+B) * C/D + E^{\wedge}F/G$

$AB +$ $* C/D + E^{\wedge}F/G$

Let $AB + = R_1$

$R_1 * C/D + E^{\wedge}F/G$

$R_1 * C/D + (^{\wedge}EF)/G$

Let $EF^{\wedge} = R_2$

$R_1 * C/D + R_2/G$

$R_1 * (CD/) + R_2/G$

Let $CD/ = R_3$

$R_1 * R_3 + R_2/G$

$R_1 * R_3 + (R_2G/)$

Let $R_2G/ = R_4$

$R_1 * R_3 + R_4$

$(R_1 R_3 *)$ $+ R_4$

Let $R_1 R_3 * = R_5$

$R_5 + R_4$

$R_5 R_4 +$

Now enter the value of R_5, R_4, R_3, R_2, R_1

$R_5 R_4 +$

$R_1 R_3 * R_4 +$

$AB + CD / * R_2 G / +$

$AB + CD / * (^{\wedge}EF G / +$

Postfix expression.

14

Prefix and Postfix using tabular form

Ex:- Convert $(A + B * C)$ into prefix and postfix using tabular form

to convert in Prefix following operation Program

- 1). Reverse the input string
- 2). Perform tabular method and find postfix Expression.
- 3). Reverse this postfix Expression string to find the Prefix.

Ex:- $A + B * C$

first to add branches
 $(A + B * C)$

Reverse string
 $(C * B + A)$

Priority
$\wedge \rightarrow$ highest
$*, / \rightarrow$ 2 highest
$+, - \rightarrow$ Lowest property.

15

Tabular Form

Symbol Scanned

(
(
(
*
B
+
A
)

Stack

(
(
(*
(*
C+
C+
-xy

Postfix Expression

C
C
CB
CB*
CB*A
CB*A+

So the postfix Expression $CB * A +$. Now reverse this Expression to get the prefix so prefix is $+ A * BC$ Prefix

Convert postfix $\Rightarrow$ Direct perform tabular form $(A + B * C)$

Symbol Scanned

(
A
+
B
*
C
)

Stack

(
(
(+
(+
(+*
(+*
xy

Postfix Expression

A
A
AB
AB
ABC
ABC*+

Postfix Expression = $ABC * +$

16

QUEUES

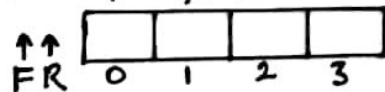
- Queue is a Non-Primitive linear data structure.
- It is an homogeneous collection of elements in which new elements are added at one end called the Rear End, and the existing element are deleted from other end called the Front End.
- The first added element will be the first to be remove from the queue. that is the reason queue is called (FIFO) First-In-First-out type list.
- In queue every insert operation Rear is incremented by one.

$$R = R + 1$$

and every deleted operation front is incremented by one.

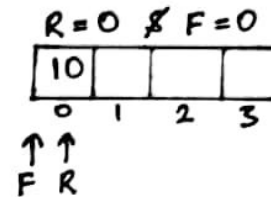
$$F = F + 1$$

Ex $\Rightarrow F = -1, R = -1$

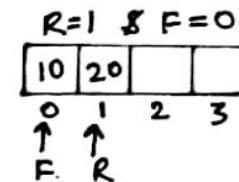


Empty queue

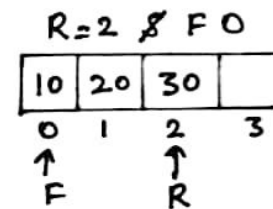
Insert 10



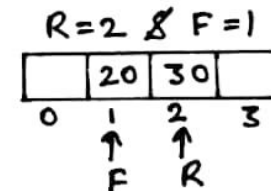
Insert 20



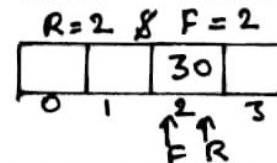
Insert 30



delete element. First delete 10



delete second element.



Operation on Queue

1). To Insert an Element in a Queue :

Algo: QINSERT [QUEUE[maxsize], Item]

Step 1: Initialization

set front = -1

set Rear = -1

Step 2: Repeat steps 3 to 5 until

Rear < maxsize - 1

Step 3: Read Item

Step 4: if front == -1 then

front = 0

Rear = 0

else

Rear = Rear + 1

Step 5: set QUEUE[Rear] = item

Step 6: Print, Queue is overflow.

2). To delete an element from the Queue:

QDELETE (Queue[maxsize], item)

Step 1: Repeat step 2 to 4 until

front >= 0

Step 2: Set item = Queue[front]

Step 3: If front == Rear

set front = -1

set Rear = -1

else

front = front + 1

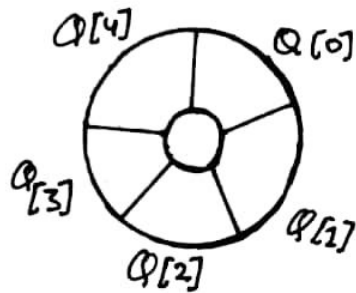
Step 4: Print, No. Deleted is, item

Step 5: Print "Queue is Empty or
underflow."

CIRCULAR QUEUE

A Circular queue is one in which the insertion of a new element is done at the very first location of the queue if the last location of queue is full.

$$F = R = -1$$



A Circular queue overcome the problem of Unutilized space in Linear queues implemented as arrays.

Circular queue has following Condition:

- 1). Front will always be pointing to the first element.
- 2). If $Front = Rear$ the queue will be empty.

3). Each time a new element is inserted into the queue the Rear is incremented by one.

$$Rear = Rear + 1$$

4). Each time an element is deleted from the queue the value of front is incremented by one.

$$Front = Front + 1$$

Insert an element in circular queue:

Algo $\Rightarrow$ QINSERT (QUEUE[maxsize], Item)

Step 1 $\Rightarrow$ If $(front == (Rear + 1) \% maxsize)$
Write queue is over flow & Exit.

Else: take the value

if $(front == -1)$

Set $front = 0$

$Rear = 0$

Else

$Rear = ((Rear + 1) \% maxsize)$

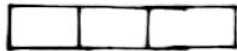
[Assign value] Queue[Rear] = value.

[End if].

Queue (Data Structure)

Operation on Queue

Ex:-



maxsize = 3

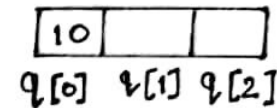
10, 20, 30, 40.

1). Front = -1 ✓ Empty queue
Rear = -1

• 3 to 5 step Repeat
Rear < maxsize - 1
-1 < 3 - 1
-1 < 2 true
 3 4 5

• Read item
 Read 10
• F == -1
 -1 == -1 true
 F = 0
 R = 0

• Set $q[0] = \text{item}$
 $q[0] = 10$



Queue

10		
----	--	--

 $q[0]$ $q[1]$ $q[2]$

• F = 0, R = 0
Rear < maxsize - 1
 0 < 3 - 1
 0 < 2 true
Read 20

• if F == -1
 0 == -1 false
 else
 R = R + 1
 R = 0 + 1
 R = 1

• $q[1] = 20$

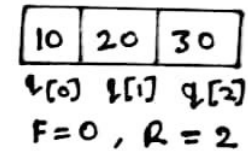
10	20	
----	----	--

 $q[0]$ $q[1]$ $q[2]$
 F = 0, R = 1

2) Rear < maxsize - 1
 1 < 3 - 1
 1 < 2 true
Read 30
 if F == -1
 0 == -1 false
 else

R = R + 1
R = 1 + 1 = 2

• Set $q[2] = 30$



case 2 Rear < maxsize - 1
 2 < 3 - 1
 2 < 2 false

• Queue is overflow.

Delete an Element In Circular Queue :

Algo : QDELETE (Queue [maxsize], Item)

1). if (front = -1)
Write queue underflow and Exit

Else: item = Queue [front]

if (front == Rear)

set front = -1

set Rear = -1

Else: front = ((front + 1) % maxsize)

[end if statement]

→ item deleted.

2). Exit.

QUEUE (Data structure)

Delete Operation On Queue.

Ex:-

10	20	30
----	----	----

 maxsize = 3

q[0] q[1] q[2]

F = 0 , R = 2

Case 1). 1). $f \geq 0$

$0 \geq 0$ true.

2) set item = q [0]

item = 10

3). $f == R$

$0 == 2$ false

Else

$F = F + 1$

$f = 0 + 1 = 1$

4). Item is deleted
10 is deleted.

	20	30
--	----	----

q[0] q[1] q[2]

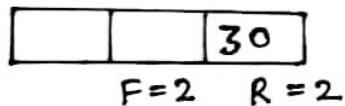
F = 1 R = 2

	20	30
--	----	----

F = 1 R = 2

Case 2).

- 1). $F \geq 0$
 $1 \geq 0$ true
- 2). $item = Q[1]$
 $item = 20$
- 3). if $F == R$
 $1 == 2$ False
else
 $F = F + 1$
 $F = 1 + 1 = 2$
- 4). Item is deleted
20 is deleted.



Case 3).

- 1). $F \geq 0$
 $2 \geq 0$ true
- 2). $item = Q[2]$
 $item = 30$
- 3). if $F == R$
 $2 == 2$ true
set $F = -1$
 $R = -1$

4). Item is deleted.



$F = -1$
 $R = -1$

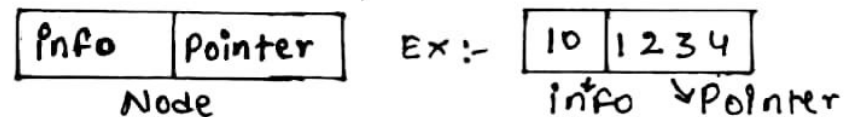
Case 4).

$F \geq 0$
 $-1 \geq 0$ False

Step 5: Queue is empty
is Underflow.

Linked Lists.

- A Linked List is a linear data structure, in which the elements are not stored at contiguous memory location.
- A Linked List is a dynamic data structure. The No. of nodes in a list is not fixed and can grow and shrink on demand.
- Each Element is called a node which has two parts.
Info part which stores the Information and Pointer which point to the next element.



Advantages of Linked Lists

1). **Linked List are dynamic data structure :**

That is, they can grow and shrink during the execution of a program.

2). **Efficient memory utilization:**

Here, memory is not pre-allocated. Memory is allocated whenever it's required. And it's deallocated (Removed) when it's no longer needed.

3). **Insertion and deletions are easier & efficient**

It provide flexibility in inserting a data item at a specified position and deletion of a data item from the given position.

4). Many complex Applications can be easily carried out with linked lists.

Operation On Linked List:

The Basic operation to be performed on the Linked Lists are :-

1). **Creation:** This operation are used to create a Linked list. In this node is created and Linked to the another node.

2). **Insertion:** This operation is used to insert a new node in the Linked list. A new node may be inserted.

- At the beginning of a Linked list.
- At the End of a Linked list.
- At the specified position in a Linked list.

3). **Deletion:** This Operation is used to delete an item (a node) from the Linked list. A node may be deleted from.

- Beginning of a Linked list.
- End of a Linked list.
- Specified position in the list.

4). **Traversing :** It is a process of going through all the nodes of a linked list from one end to the other end.

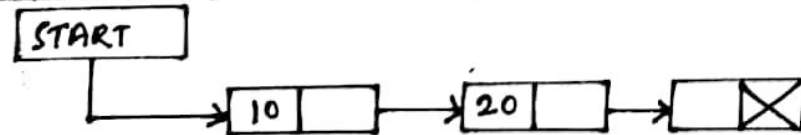
5). **Concatenation :** It's the process of the joining the second list to the end of the first list.

6). **Display :** This operation is used to print each and every nodes information.

Types of Linked List.

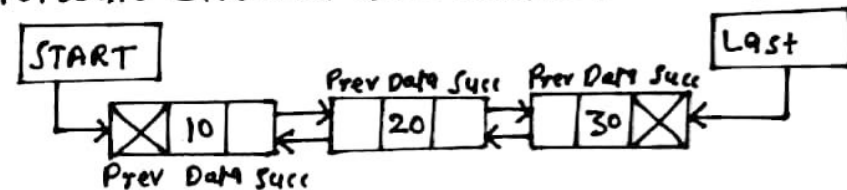
- Basically, there are four type of linked list.

1). **Singly-Linked List :** It's one in which all nodes are linked together in some sequential manner. it is also called linear linked list.

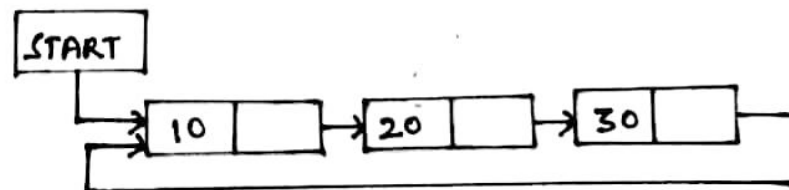


31

2). **Doubly-Linked List :** It's one in which all nodes are linked together by multiple links which help in accessing both the successor node (next node) and predecessor node (previous node) within the list. This help to traverse the list in the forward direction and backward direction.

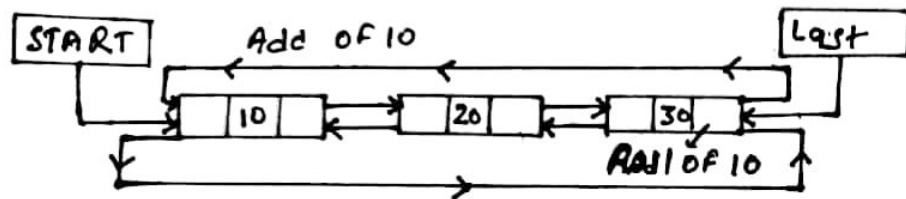


3). **Circular Linked List :** It's one which has no beginning and no end. A singly linked list can be made a circular linked list by simply sorting the address of the very first node in the link field of the last node.



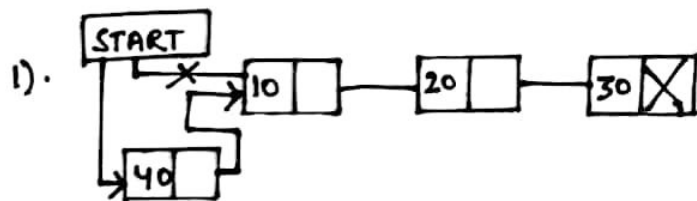
32

4). **Circular doubly Linked List:** It's one which both the successor pointer and predecessor pointer in a circular manner.

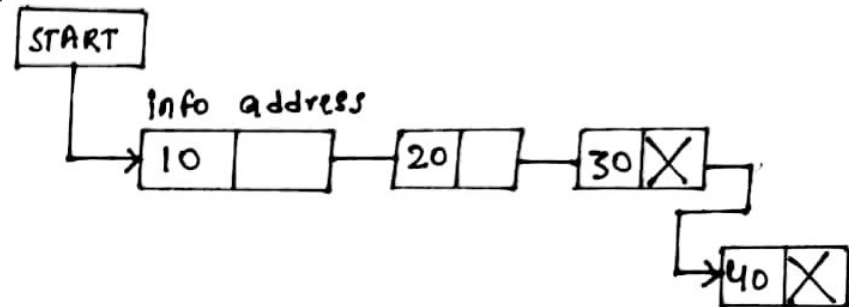


Inserting of Nodes in Linked List

- 1). Inserting at the beginning of the list.
- 2). Inserting at the end of the list.
- 3). Inserting at the specified position within the list.



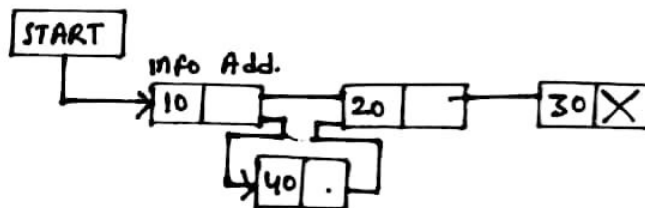
2).



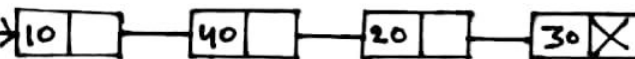
START



3).



START



LINKED LIST

Inserting a node at the Beginning In Linked List.

Algorithm $\Rightarrow$

INSERT_FIRST (START, ITEM)

Step 1: [Check for overflow]

IF PTR = NULL then
Print overflow
Exit

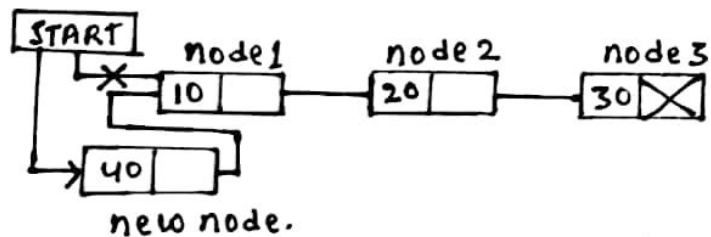
Else

PTR = (Node *) malloc (size of (Node))
// Create new node from memory and
assign its address to PTR.

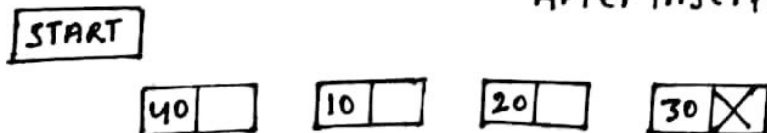
Step 2: Set PTR $\rightarrow$ INFO = Item.

Step 3: Set PTR $\rightarrow$ Next = START

Step 4: Set START = PTR



After Insertion



35

LINKED LIST

Insert a Node at the End in singly linked.

Algorithm $\Rightarrow$

INSERT_LAST (START, ITEM)

Step 1: Check for overflow

IF PTR = NULL then
Print overflow

Exit

PTR = (Node *) malloc (size of
(Node));

Step 2: Set PTR $\rightarrow$ Info = Item ;

Step 3: Set PTR $\rightarrow$ Next = NULL ;

Step 4: If start = NULL and then
set START = PTR ;

Else,

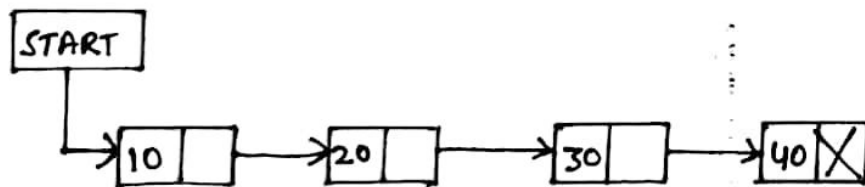
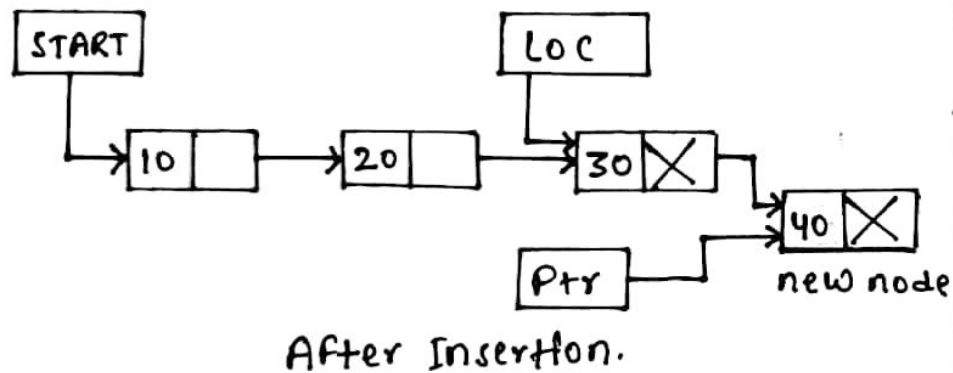
Step 5: Set Loc = start

36

Step 6: Repeat step 7 until $\text{Loc} \rightarrow \text{Next} \neq \text{Null}$

Step 7: Set $\text{Loc} = \text{Loc} \rightarrow \text{Next}$;

Step 8: Set $\text{Loc} \rightarrow \text{Next} = \text{Ptr}$;



LINKED LIST

Inserting a node at the specific position in singly linked list

Algorithm $\Rightarrow$

Insert_Location (START, Item, Loc)

Step 1: Check for overflow

If $\text{Ptr} = \text{Null}$ then

Print overflow

Exit

Else

$\text{Ptr} = (\text{Node}^*) \text{malloc}(\text{size of}(\text{Node}))$

Step 2: Set $\text{Ptr} \rightarrow \text{Info} = \text{Item}$

Step 3: If $\text{start} = \text{Null}$ then

Set $\text{start} = \text{Ptr}$

Set $\text{Ptr} \rightarrow \text{Next} = \text{Null}$

Step 4: Initialize the counter i and pointers

Set $i = 0$

Set $\text{temp} = \text{start}$

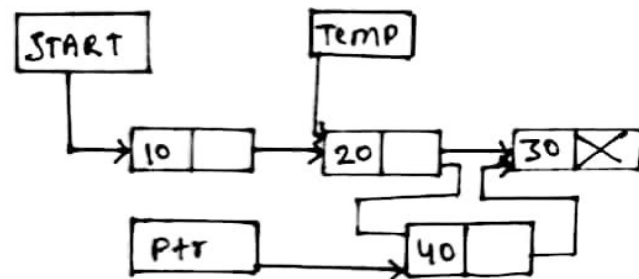
Step 5: Repeat steps 6 and 7 until $I < \text{Loc}$

Step 6: Set $\text{temp} = \text{temp} \rightarrow \text{next}$

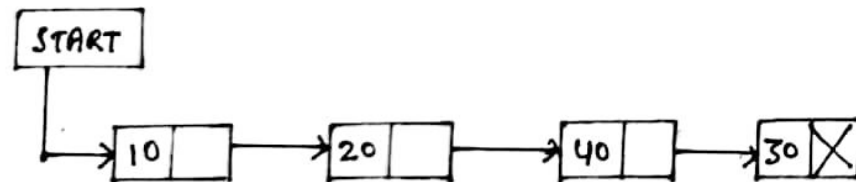
Step 7: Set $I = I + 1$

Step 8: Set $\text{Ptr} \rightarrow \text{Next} = \text{temp} \rightarrow \text{Next}$

Step 9: Set $\text{temp} \rightarrow \text{Next} = \text{Ptr}$



After Insertion



Deleting Node in Linked List

Deleting a node from the Linked List has three instances.

1 $\Rightarrow$ Deleting the first node of the Linked List.

2 $\Rightarrow$ Deleting the last node of the Linked List.

3 $\Rightarrow$ Deleting the node from specified position of the Linked List.

Linked List Deleting Nodes

Deleting the first Node in singly Linked List

Algorithms $\Rightarrow$

Deleted first (START)

Step 1: Check for underflow.

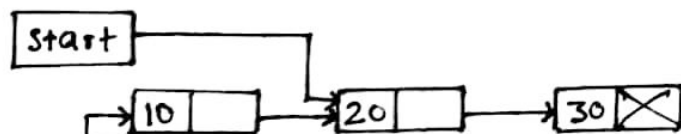
If start = NULL, then
print linked list empty
Exit.

Step 2: Set PTR = START

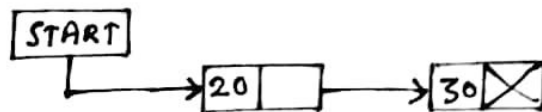
Step 3: Set START = START $\rightarrow$ Next

Step 4: Print Element deleted is PTR $\rightarrow$ info

Step 5: free (PTR)



After deletion



Linked List Deleting Nodes

Deleting the last node in singly linked List

Algorithm $\Rightarrow$

Deleting (START)

Step 1: Check for Underflow

If start = NULL then
Print linked list is empty
Exit

Step 2: If start $\rightarrow$ Next = NULL then

Set PTR = start

Set start = NULL

Print element deleted is = PTR $\rightarrow$ Info
free (PTR)

End If

Step 3: Set PTR = START

Step 4: Repeat step 5 and 6 until

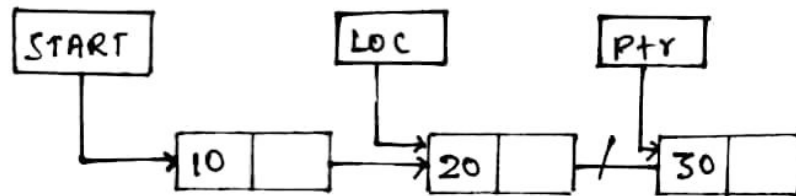
PTR $\rightarrow$ Next $\neq$ NULL.

Step 5: Set Loc = PTR

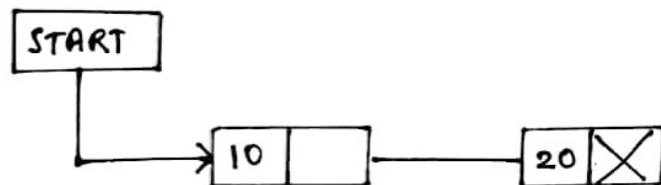
Step 6: Set $PTR = PTR \rightarrow Next$

Step 7: Set $LOC \rightarrow Next = NULL$

Step 8: Free (PTR).



After deletion



LINKED LIST DELETING NODES

Deleting the Nodes from Specified position in Singly Linked list.

Algorithm $\Rightarrow$

Delete - Location (START, LOC)

Step 1: Check for underflow

if $PTR = NULL$ then

Print underflow

Exit

Step 2: Initialize the counter I and pointers.

Set $I = 0$;

Set $PTR = start$;

Step 3: Repeat step 4 to 6 until

$I < LOC$

Step 4: Set $temp = PTR$

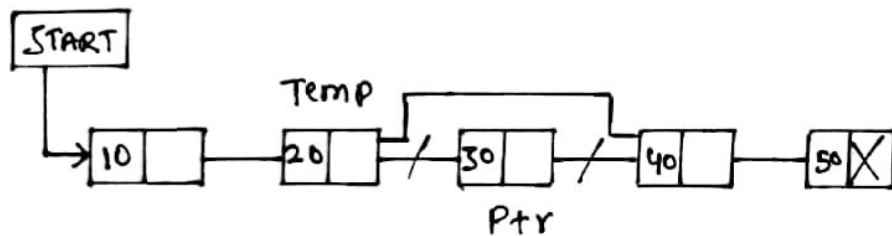
Step 5: Set $PTR = PTR \rightarrow Next$

Step 6: Set $I = I + 1$.

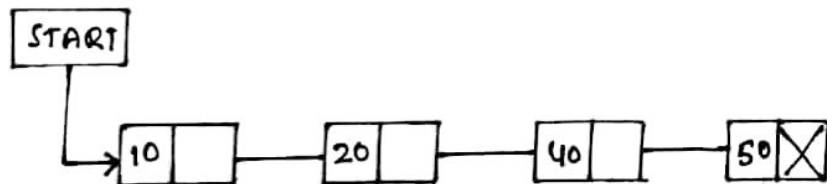
Step 7: Print Element deleted is
= Ptr $\rightarrow$ Info

Step 8: Set Temp $\rightarrow$ Next = Ptr $\rightarrow$ Next

Step 9: free (Ptr)



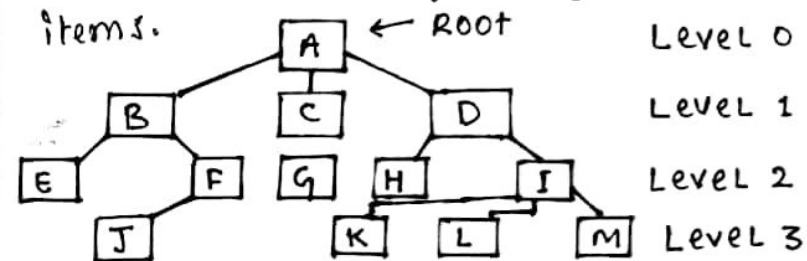
After deletion.



TREES IN DATA STRUCTURE

Tree: A Tree is a non-linear data structure in which items are arranged in a sorted sequence.

- It is used to represent hierarchical relationship existing amongst several data items.



Tree Terminology: Tree has different

terminology such as:-

- 1). **Root:** It is specially designed data item in a tree. It is the first in the hierarchical arrangement of data item.
- 2). **Node:** Each data item in a tree is called a node. In the given tree there are 13 nodes such as:- A, B, C, D, E, F, G, H, I, J, K, L, M.

3). **Degree of a node:** It is the no. of subtrees of a node in a given tree.

The degree of A = 3

The degree of C = 1

The degree of L = 0

4). **Degree of a tree:** It is the maximum degree of nodes in a given tree. In the given tree the node A and node I has maximum degree (3). So the degree of tree is 3.

5). **Terminal node:** A node with degree zero is called terminal node. In given tree - E, J, G, H, K, L and M are terminal node.

6). **Non-Terminal Node:** Any node whose degree is not zero is called non-terminal node. In given tree - A, B, C, D, F, I are non-terminal node.

7). **Siblings:** The child nodes of a given parent node are called Siblings. They are also called brothers.

In the given table.

- B, C, D are siblings of parent node A.
- H & I are siblings of parent node D.

47

8). **Level:** The entire tree structure is Levelled in such a way that the root node is always at level 0.

9). **Edge:** It is a connecting line of two nodes. that is, the line drawn from one node to another node is called an Edge.

10). **Path:** It is a sequence of consecutive edges from the source node to the destination node. In the given tree the path b/w A and J is as.

(A, B) (B, F) and (F, J)

A → B → F → J

11). **Depth:** It is the maximum level of any node in a given tree. In the given tree, the root node A has the maximum level.

12). **Forest:** It is a set of disjoint trees. In a given tree if you remove its root node then it becomes a forest. In the given tree, there is forest with three tree. such as.

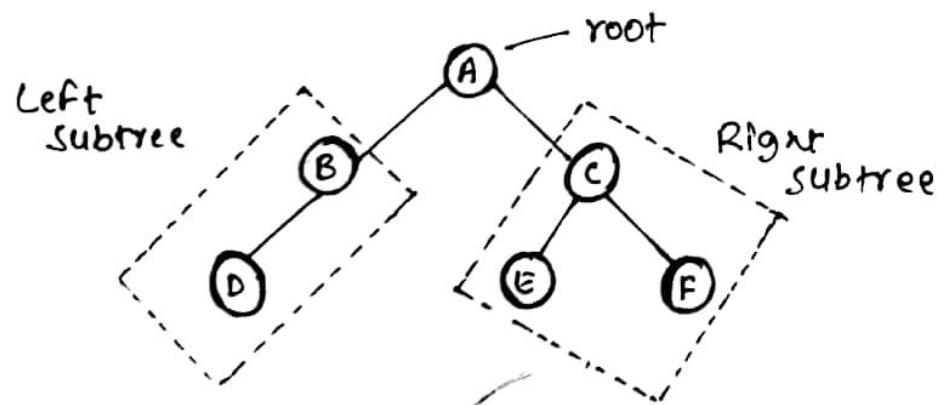
After removing root A. forest is



48

BINARY TREES

- Binary tree is a finite set of data item which is either empty or consists of a single item called root and two disjoint binary tree called the Left Subtree and Right Subtree.
- In Binary tree, Every node can have maximum of 2 children which are known as Left child and Right child.

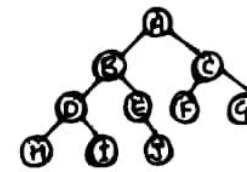


TYPES OF BINARY TREES

- 1). **Full Binary tree:** A Binary tree is full if every node has 0 or 2 child.

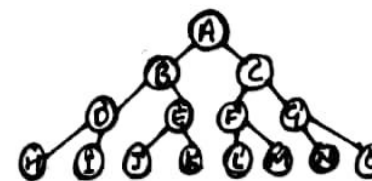


- 2). **Complete Binary tree:** A Binary tree is complete binary tree if all levels are completely filled except possibly the last level and the last level has all keys as left as possible.



Level 0	$2^0 = 1$
Level 1	$2^1 = 2$
Level 2	$2^2 = 4$
Level 3	$2^3 = 8$

- 3). **Perfect Binary Tree:** A tree in which all internal nodes has two children and all leaves are at same level.



in which all level has 2^n child.	
Level 0	$2^0 = 1$
Level 1	$2^1 = 2$
Level 2	$2^2 = 4$
Level 3	$2^3 = 8$

Traversal of a Binary Tree

It is a way in which each node in the tree is visited exactly once in a systematic manner.

There are three ways which we use to traverse a tree -

- | | | | |
|--------------------------|------|------|-------|
| | Node | Left | Right |
| | ↑ | ↑ | ↑ |
| 1 - Pre Order traversal | (N | , L | , R) |
| 2 - In order traversal | (L | , N | , R) |
| 3 - Post order traversal | (L | , R | , N) |

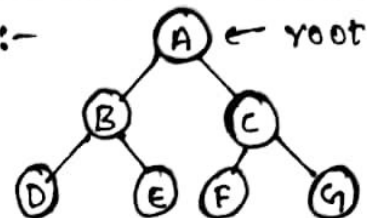
1- **Pre order Traversal:** In this Traversal method, the root(N) node is visited first, then the Left(L) subtree and finally the right(R) subtree.

Algorithm $\Rightarrow$

Until all nodes are traversed -

- Step 1: Visit root node.
- Step 2: Recursively traverse left subtree.
- Step 3: Recursively traverse Right subtree.

Ex:-



Pre - order traversal is $\rightarrow$
A, B, E, D, C, F, G.

2) **Inorder Traversal:** In this traversal method, the Left(L) subtree is visited first, then the root(N) and later the right(R) subtree.

Algorithm $\Rightarrow$

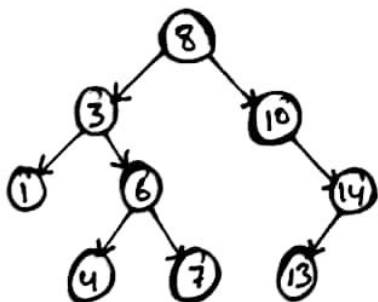
Until all nodes are traversed -

- Step 1: Recursively traverse left subtree.
- Step 2: Visit root node.
- Step 3: Recursively traverse Right subtree.

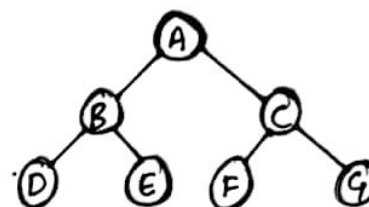
Binary Search tree (BST)

- Binary search tree is a node-based binary tree data structure which has the following Rules:-

- 1). The value of the key in the left child or left subtree is less than the value of root.
- 2). The value of the key in the right child or right subtree is more than or equal to the root.
- 3). The right and left subtree each must also be a binary search tree (BST).



Ex:-



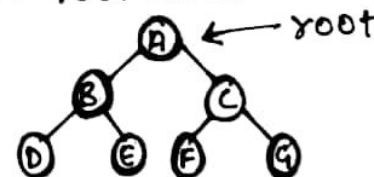
Inorder Traversal is —
D, B, E, A, F, C, G.

- 3). **Post-Order Traversal:** In this method the root node is visited last, hence the name first we traverse left(L) subtree, then the right(R) subtree and finally the root(N) node.

Algorithm ⇒

- Step 1: Recursively traverse left subtree.
- Step 2: Recursively traverse right subtree.
- Step 3: visit root node.

Ex:-



post order Traversal is —
D, E, B, F, G, C, A

DIFFERENCE BETWEEN STACK and QUEUE

STACK

- 1). It represents the collection of elements in Last In first out (LIFO) order.
- 2). Objects are inserted and removed at the same end called Top of stack (Tos).
- 3). Insert operation is called push operation.
- 4). Delete operation is called POP operation.
- 5). In stack there is no wastage of memory space.
- 6). Plate counter at marriage Reception is an Example of stack.

QUEUE

- 1). It represents the collection of elements in first In first out (FIFO) order.
- 2). Object are inserted and removed from different ends called front and rear ends.
- 3). Insert Operation is called Enqueue Operation.
- 4). Delete Operation is called Dequeue operation.
- 5). In Queue there is a wastage of memory space.
- 6). Students standing in a line at fees counter is an example of Queue.

DIFFERENCE BETWEEN SINGLY & DOUBLY LINKED LIST

SINGLY LINKED LIST

- 1). Singly Linked List has nodes with data field and next link field (forward link).

Ex:-

Data	next
------	------

- 2). It allows traversal only in one way.
- 3). It requires one list pointer variable (start).
- 4). It occupies less memory.
- 5). Complexity of Insertion and Deletion at known position is $O(n)$

DOUBLY LINKED LIST

- 1). Doubly Linked List has nodes with data field and two pointer field. (Backward and forward link).

Ex:-

Previous	Data	Next
----------	------	------

- 2). It allows a two way traversal.
- 3). It requires two list pointer variable (start and last).
- 4). It occupies more memory.
- 5). Complexity of Insertion and Deletion at known position is $O(1)$.

DIFFERENCE BETWEEN LINEAR & NON-LINEAR DATA STRUCTURE

LINEAR DATA STRUCTURE

- 1). In this data structure, The elements are organized in a sequence such as:-
Ex:- Array, stack, queue etc..
- 2). In Linear data structure single level is involved.
- 3). It is easy to implement.
- 4). Data elements can be traversed in a single run only.
- 5). Memory is not utilized in a efficient way.
- 6). Application of Linear D.S. are mainly in Application software development.

NON-LINEAR DATA STRUCTURE

- 1). In this data structure data is organized without any sequence.
Ex:- Tree, Graph etc.
- 2). In Non-Linear Data structure multiple levels are involved.
- 3). It is difficult to implement.
- 4). Data elements can't be traversed in a single run only.
- 5). Memory utilization in an efficient way.
- 6). Application of non-Linear D.S. are in Artificial Intelligence and Image Processing.

DIFFERENCE BETWEEN ARRAY AND LINKED LIST

ARRAY

- 1). Size of an Array is fixed.
- 2). Array is a collection of Homogeneous (similar) data type.
- 3). Memory is allocated from stack
- 4). Array work with static data structure.
- 5). Elements are stored in contiguous memory location.
- 6). Array elements are independent to each other.
- 7). Array take more time.
(Insertion & Deletion)

61

LINKED LIST

- 1). Size of a list is not fixed.
- 2). Linked list is a collection of node (data & address)
- 3). Memory is allocated from heap.
- 4). Linked list work with dynamic data structure.
- 5). Elements can be stored anywhere in the memory.
- 6). Linked list elements are depend to each other.
- 7). Linked - list take less time.
(Insertion & Deletion)

62

DIFFERENCE BETWEEN TREE AND GRAPH

TREE

1). Tree is a collection of nodes and Edges.
EX:- $T = \{\text{node, Edges}\}$

2). There is a unique node called root in tree.

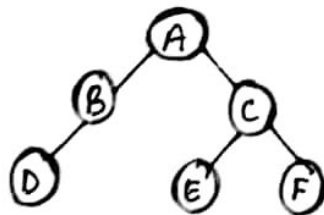
3). There will not be any cycle/loops.

4). Represents data in the form of a tree structure, in a hierarchical manner.

5). In tree only one path between two nodes.

6). In this preorder, In order and preorder Traversal.

EX:-



GRAPH

1). Graph is a collection of vertices/nodes and Edges.

EX:- $G = \{V, E\}$

2). There is no unique node.

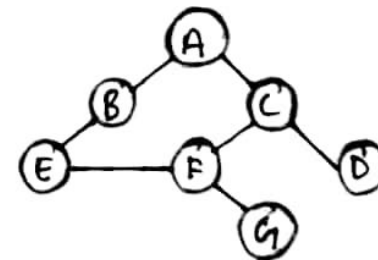
3). There can be loops/cycle.

4). Represents data similar to a network.

5). In Graph One or more than one path between two nodes.

6). In this BFS and DFS traversal.

EX:-



Data Structure & Algorithm

Part - 2

ATUL KUMAR (LINKEDIN)

INSTAGRAM - @codeatul

? What is Sorting? Discuss it's types.

✓ Sorting is a method to arrange N elements of array in a particular format either ascending or descending Order.

Types :-

- Bubble Sort
- Selection Sort
- Insertion Sort
- Quick Sort
- Merge Sort
- Heap Sort
- Radix Sort
- Shell Sort
- tree Sort
- Bucket Sort

Example :-

int a[5] = {10, 12, 8, 100, 7}

ascending → 7, 8, 10, 12, 100

descending → 100, 12, 10, 8, 7

? What is Bubble Sort? Write algorithm & procedure.

✓ Bubble Sort:- Bubble sort arrange N number of array elements by placing the biggest element on proper position. It always arrange the data in descending order.

ex $\rightarrow$ $\text{int } a[5] = \{ 7, 6, 3, 4, 1 \}$
 $= 1, 3, 4, 6, 7.$

Procedure:-

(i) $\begin{array}{ccccc} 7 & 6 & 3 & 4 & 1 \\ 6 & 7 & 3 & 4 & 1 \\ 6 & 3 & 7 & 4 & 1 \\ 6 & 3 & 4 & 7 & 1 \\ 6 & 3 & 4 & 1 & \boxed{7} \rightarrow 7 \end{array}$

(ii) $\begin{array}{cccc} 6 & 3 & 4 & 1 \\ 3 & 6 & 4 & 1 \\ 3 & 4 & 6 & 1 \\ 3 & 4 & 1 & \boxed{6} \rightarrow 6 \end{array}$

(iii) $\begin{array}{ccc} 3 & 4 & 1 \\ \times 4 & 3 & \\ 3 & 4 & 2 \\ 3 & 1 & \boxed{4} \rightarrow 4 \end{array}$

(iv) $\begin{array}{cc} 3 & 1 \\ 1 & \boxed{3} \rightarrow 3 \end{array}$

(v) $\begin{array}{c} 1 \end{array}$

$\boxed{1 \ 3 \ 4 \ 6 \ 7}$

6 3 4 1 7

6 3 4 7 1

6 3 7 4 1

6 7 3 4 1

Algorithm for Bubble Sort :-

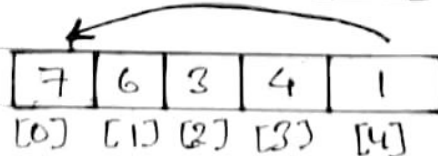
Step 1: Begin

Step 2: input $a[5]$ ~~0/-4~~ {7, 6, 3, 4, 1}Step 3: Set $i \leftarrow 3$ Step 4: Repeat Step (4) & (9) while ($i \geq 0$)Step 5: Set $j \leftarrow 0$ Step 6: Repeat Step (7) & (8) while ($j \leq i$)Step 7: if $a[j] > a[j+1]$ thenSet $temp \leftarrow a[j]$ $a[j] \leftarrow a[j+1]$ $a[j+1] \leftarrow temp$ Step 8: $j \leftarrow j+1$ Step 9: $i \leftarrow i-1$ Step 10: print $a[5]$

Step 11: exit.

? What is Selection Sort? Algorithm & procedure.

✓ Selection Sort arrange N elements of array by placing the smallest element in proper position in case of ascending order arrangement.

Procedure :- $int a[5] = \{7, 6, 3, 4, 1\}$ 

1 6 3 4 7

1 3 6 4 7

1 3 4 6 7

ascending Order.

Algorithm:-

Step 1: Begin

Step 2: Input $a[5] = \{ 7, 6, 3, 4, 1 \}$

Step 3: set $i \leftarrow 0$ 1

Step 4: Repeat step (5) to (10) while $(i < 4)$

Step 5: set $m \leftarrow a[i]$, $i \leftarrow i + 1$

Step 6: Repeat step (7) to (8) while $(j < 5)$

Step 7: if $m > a[j]$ then

set $m \leftarrow a[j]$

set $loc \leftarrow j$

Step 8: $j \leftarrow j + 1$

Step 9: if $a[loc] < a[i]$ then

set $temp \leftarrow a[loc]$

$a[loc] \leftarrow a[i]$

$a[i] \leftarrow temp$

Step 10: Set $i \leftarrow i + 1$

Step 11: print $a[5]$

Step 12: exit.

$i = 0$

$j = 1$

$m = 7$

$loc = 1$

? What is Insertion Sort? Write algorithm and Procedure.

Q Insertion sort arrange N elements of array by inserting particular item in a particular place such a way that the item are in sorted order.

Procedure:-

$a[0] \ a[1] \ a[2] \ a[3] \ a[4]$
 $\overbrace{7 \ 6 \ 3 \ 4 \ 1}$

$6 \ 7 \ 3 \ 4 \ 1$

$6 \ 3 \ 7 \ 4 \ 1$

$3 \ 6 \ 7 \ 4 \ 1$

$3 \ 6 \ 4 \ 7 \ 1$

$3 \ 4 \ 6 \ 7 \ 1$

$3 \ 4 \ 6 \ 1 \ 7$

$3 \ 4 \ 1 \ 6 \ 7$

$3 \ 1 \ 4 \ 6 \ 7$

$\boxed{1} \ \boxed{3} \ \boxed{4} \ \boxed{6} \ \boxed{7}$

Algorithm.

Step 1: Begin

Step 2: Input $a[5] = \{ \textcircled{7}, \textcircled{6}, 3, 4, 1 \}$

Step 3: Set $i \leftarrow 1$

Step 4: Repeat step (5) to (9) while $(i < 5)$

Step 5: Set $i \leftarrow 0$

Step 6: Repeat step ⑦ to ⑧ while $j \geq 1$

Step 7: if $a[j-1] > a[j]$ then

set $temp \leftarrow a[j-1]$

$a[j-1] \leftarrow a[j]$

$a[j] \leftarrow temp.$

Step 8: $j \leftarrow j-1$

Step 9: $i \leftarrow l+1$

Step 10: Print $a[5]$

Step 11: Exit.

? What is merge sort? Write algorithm & Procedure.

✓ Merge Sort is a sort algorithm that splits the items of array to be sorted Order into two group, recursively sort each group and merges then into a final sorted sequence.

Procedure:-

7, 1, 8, 3, 9, 2, 10, 12

1, 7, 3, 8, 2, 9, 10, 12

1, 3, 7, 8, 2, 9, 10, 12

1, 2, 3, 7, 8, 9, 10, 12

Algorithm:-

Step 1: Begin

Step 2: Divides all the items of array into two items pairs. If single item remains make a single group.

Step 3: Sort each of the pairs.

Step 4: Merge two pairs into a single pair.

Step 5: Sort the merge pair.

Step 6: Repeat the above steps until the all elements of array are not sorted.

Step 7: exit.

? What is Quick Sort? Write algorithm & Procedure.

✓

The Quick Sort algorithm items to separates the list of elements into two parts and then sort each part recursively.

It use divide and conquer method. In this method the partition of list is performed based on the element called pivot element.

The list is divided into two partition such that all elements to the left of pivot are smaller than the pivot and all elements of right of pivot are greater than or equal to pivot.

Procedure :-

Pivot element	(50)	3	1	60	65	45	90	13	67
	13	3	1	60	65	45	90	(50)	67
	13	3	1	(50)	65	45	90	60	67
	13	3	1	45	65	(50)	90	60	67
	13	3	1	45	(50)	65	90	60	67
Pivot	(13)	3	1	45	50	(65)	90	60	67
	1	3	(13)	45		60	90	(65)	67
	(1)	3				60	(65)	90	67
		3						(90)	67
								67	90

1	3	13	45	50	60	65	67	90
---	---	----	----	----	----	----	----	----

Algorithm :-

Step 1: Begin.

Step 2: Select the start element of array as a pivot element.

Step 3: Scan and find smallest element from right side of array.

Step 4: Interchange both elements.

Step 5: Scan and find the biggest element from left side of array.

Step 6: Repeat the above process until all the elements of left side are smaller and right side elements are greater than pivot ~~table~~ element.

Step 7: Now, we have two sublist are available, apply same process on each sublist until the all elements of array are not started.

Step 8: exit.

Linked List

? What is Linked list? Discuss with its types.

✓ Linked List is the linear data structure where data are not stored sequentially inside the computer memory but they are link with each other by the help of address.

Example:- Data - MILAN

add	Data	Link	
1 →	①	④	
2			
3 →	N	N-1	End.
4 →	L	8	
5			
6			
start → 7	M	①	MILAN
8	①	③	

Types :-

- Singly Linked list
- doubly Linked list
- Circular linked list

?

What is Singly linked list? Explain with algorithm.

✓

It is linear collection of data item called node, where each node has divided into two part i.e. "data part & link part", data part store the data item and link part store the address of next node.

Linked list start with a special pointer called start pointer and terminated with Null pointer.

add	Data	link	Data → MILAN
1	I	4	
2			
3	N	Null	
4	L	8	
5			
6			
Start → 7	M	1	
8	(A)	3	<u>MILAN</u>

Algorithm:- Step 1: Begin $i = 7$ Null
 Step 2: Set $i \leftarrow st$ $3 \ 8 \ 4 \ 1 \ 7 \neq \text{Null}$
 Step 3: Repeat Step (4) & (5) while $i \neq \text{Null}$
 Step 4: Print Data[i]
 Step 5: $i \leftarrow \text{link}[i]$
 Step 6: exit

? W.A.A. to insert an item at the beginning of Singly linked list.

✓

- Step 1: Begin
 Step 2: IF $fr = \text{Null}$ then, print Overflow and exit.
 Step 3: input new item
 Step 4: Set $new \leftarrow fr$, $fr \leftarrow \text{link}[fr]$
 Step 5: $\text{Data}[new] \leftarrow \text{item}$
 Step 6: $\text{link}[new] \leftarrow st$
 Step 7: $st \leftarrow new$
 Step 8: exit

Data - ABCD

Data $\rightarrow$ *ABCD

add		Data	link
$st \rightarrow 1$	1	A	2
	2	B	3
	3	C	4
	4	D	Null
$fr \rightarrow 5$	5		6
	6		7
	7		8
	8		Null

add		Data	link	
$st \rightarrow 1$	1	A	2	
	2	B	3	
	3	C	4	
	4	D	Null	
$new\ fr \rightarrow 5$	5	X	6	$\leftarrow st$
$fr \rightarrow 6$	6		7	
	7		8	
	8		Null	

? W.A.A. to insert an item at the ending of Singly linked list.

✓

- Step 1: Begin.
 Step 2: if $fr = \text{Null}$ then
 Print Overflow and exit.
 Step 3: input new item.

Step 4: $\text{set new} \leftarrow \text{fr}$, $\text{fr} \leftarrow \text{link}[\text{fr}]$

Step 5: $\text{set loc} \leftarrow \text{st}$ 1, 2, 3, 4

Step 6: Repeat Step (a) while $\text{link}[\text{loc}] \neq \text{Null}$
 (a) $\text{loc} \leftarrow \text{link}[\text{loc}]$

Step 7: $\text{data}[\text{new}] \leftarrow \text{item}$

Step 8: $\text{link}[\text{new}] \leftarrow \text{Null}$

Step 9: $\text{link}[\text{loc}] \leftarrow \text{new}$

Step 10: exit

Data $\rightarrow$ ABCD

$\text{loc} \rightarrow 1, 2, 3, 4$
 Data - ABCDx

add	Data	link
st $\rightarrow$ 1	A	2
2	B	3
3	C	4
4	D	Null
fr $\rightarrow$ 5		6
6		7
7		8
8		Null

add	Data	link
st $\rightarrow$ 1	A	2
2	B	3
3	C	4
4	D	Null
new fr $\rightarrow$ 5		Null
fr $\rightarrow$ 6		7
7		8
8		Null

? What is Doubly linked list? With algorithm...

✓ It is the linear collection of data item called node where each node has divided into three part i.e. data part, previous part and next part. Data part store the data item, previous part store address of previous node and next part store address of next node. It start with special pointer called first pointer and ending with last pointer.

add	Previous	Data	next
first → 1	Null	A	2
2	1	N	3
3	2	I	4
4	3	S	5
Last → 5	4	H	Null
6			7
7			8
8			Null

① Algorithm for forward traversing:

Step 1: Begin

Step 2: Set $i \leftarrow \text{first}$

Step 3: Repeat Step ④ & ⑤ while $i \neq \text{Null}$

Step 4: Print data $[i]$

Step 5: $i \leftarrow \text{next}[i]$

Step 6: Exit

ANISH

2 3 4 5 Null

add	Previous	data	next
first → 1	Null	A	2
2	1	N	3
3	2	I	4
4	3	S	5
Last → 5	4	H	Null
6			
7			
8			

② Algorithm for backward traversing.

Step 1: Begin

Step 2: Set $i \leftarrow \text{last}$

~~if~~ ~~Null~~ $\neq \text{Null}$

Step 3: Repeat step ④ & ⑤ While $i \neq \text{Null}$

Step 4: print data $[i]$

H S I N A

Step 5: $i \leftarrow \text{previous}[i]$

~~if~~ ~~Null~~

Step 6: Exit

? W.A.A. to insert an element at the beginning of doubly linked list?

✓

Algorithm:-

Step 1: Begin

Step 2: if $fr = \text{Null}$ then

print Overflow & exit.

Step 3: Input new item.

Step 4: Set $\text{new} \leftarrow fr$, $fr \leftarrow \text{next}[fr]$

Step 5: $\text{data}[\text{new}] \leftarrow \text{item}$.

Step 6: $\text{next}[\text{new}] \leftarrow \text{first}$.

Step 7: $\text{previous}[\text{new}] \leftarrow \text{last}$

Step 8: $\text{first} \leftarrow \text{new}$

Step 9: exit

①

	Previous	data	next
first → 1	Null	A	2
2	1	B	3
3	2	C	4
Last → 4	3	D	Null
fr → 5			6
6			7
7			8
8			Null

F → ABCD
B → DCBA

②

	Previous	data	next
first → 1	Null	A	2
2	1	B	3
3	2	C	4
Last → 4	3	D	Null
new fr → 5	4	X	6
fr → 6			7
7			8
8			Null

F → XABCD
B → XDCBA

? What is Circular linked list? And its types.

✓ The Circular linked list is the variation of linked list where the first node point to last node and last node point to first node.

add	Data	link
st → ①	A	2
②	B	③
③	C	4
④	D	①
5		
6		

① ABCD - - - -

② ③ ④ - - - -

add	Data	link
st → ①	A	②
2	B	3
3	C	4
4	D	Null
5		
6		

terminate.

[ABCD]

types:-

- Singly Circular linked list
- Doubly Circular linked list.

?

Circular Singly linked list:-

✓

If a last node of Singly linked list hold the address of start node then the linked list is called Circular Singly linked list.

Ex -

add	Data	link
Start → 1	A	2
2	(B)	3
3	C	4
Last → 4	D	1
5		

ABCDABCD.....

add	Data	link
1	A	2
2	B	3
3	C	4
4	(D)	Null
5		
6		

end

ABCD

Algorithm:- Step 1: BeginStep 2: Set $i \leftarrow st$ Step 3: Print Data [i] A B C DStep 4: $i \leftarrow link[i]$ $2 \neq 3$ $3 \neq 4$ $4 \neq 1$ Step 5: go to step (3) while $i \neq st$ Step 6: exit. $1 \neq 1$ X

?

W.A.A. to Print Content of C.S.L. more than OneAlgorithm- time.

Step 1: Begin

Step 2: Set $i \leftarrow st$, $j \leftarrow 1$ Step 3: Repeat step (4) to (7) while $j \leq 2$ X^2

Step 4: Print data[i]

Step 5: $i \leftarrow \text{link}[i]$

Step 6: if $i = \text{st}$ then

Step 7: $j \leftarrow j + 1$

Step 8: exit

add	Data	link
st $\rightarrow$ 1	A	2
2	B	3
3	C	4
4	D	1
5		

A B C D A B C D

? What is Circular doubly linked list?

? W.A.A. for forward and backward traversing of circular doubly linked list.

Q Circular doubly linked list:-

IF a last node of doubly linked list held the address of first node and first node of doubly linked list hold the address of last node then, the linked list is called Circular doubly linked list.

add	Previous	data	next	
First $\rightarrow$ 1	Null	A	2	
2	1	B	3	
3	2	C	4	ABCD
Last $\rightarrow$ 4	3	D	Null	DCBA
5				end.

?

W.A.A. for forward traversing of Circular doubly linked list.

✓

Algorithm:-

add	Previous	data	next
First → ①	4	(A)	②
②	1	(B)	③
③	2	(C)	④
last → ④	3	(D)	①
5			
6			

Step 1: Begin $i = \text{first}$

Step 2: Set $i \leftarrow \text{first}$

Step 3: print data [i] ABC $i \neq 1$

Step 4: $i \leftarrow \text{next}[i]$ $A \neq \text{first}$

Step 5: go to step ③ while $i \neq \text{first}$

Step 6: exit

?

W.A.A. for backward traversing of Circular doubly linked list.

✓

Algorithm

add	Previous	data	next
First → 1	4	↑ A	2
2	1	B	3
3	2	C	4
last → 4	3	D ↓	1
5			
6			

Step 1: Begin

Step 2: Set $i \leftarrow \text{last}$ 4

Step 3: print data $[i]$ DCBA

Step 4: $i \leftarrow \text{previous}[i]$

Step 5: go to step (3) while $i \neq \text{last}$,
 $3 \neq 4$
 $2 \neq 4$
 $1 \neq 4$
 $0 \neq 4$

Step 6: Exit

?

Difference between Array and linked list.

✓

Array

Linked list

(i) Array is a collection of homogenous (Similar) data type.

(i) Linked list is a collection of node (data & address).

(ii) Array elements are stored in continuous memory location.

(ii) Linked list elements can be stored anywhere in memory.

(iii) Array work with static data structure.

(iii) Linked list work with dynamic data structure.

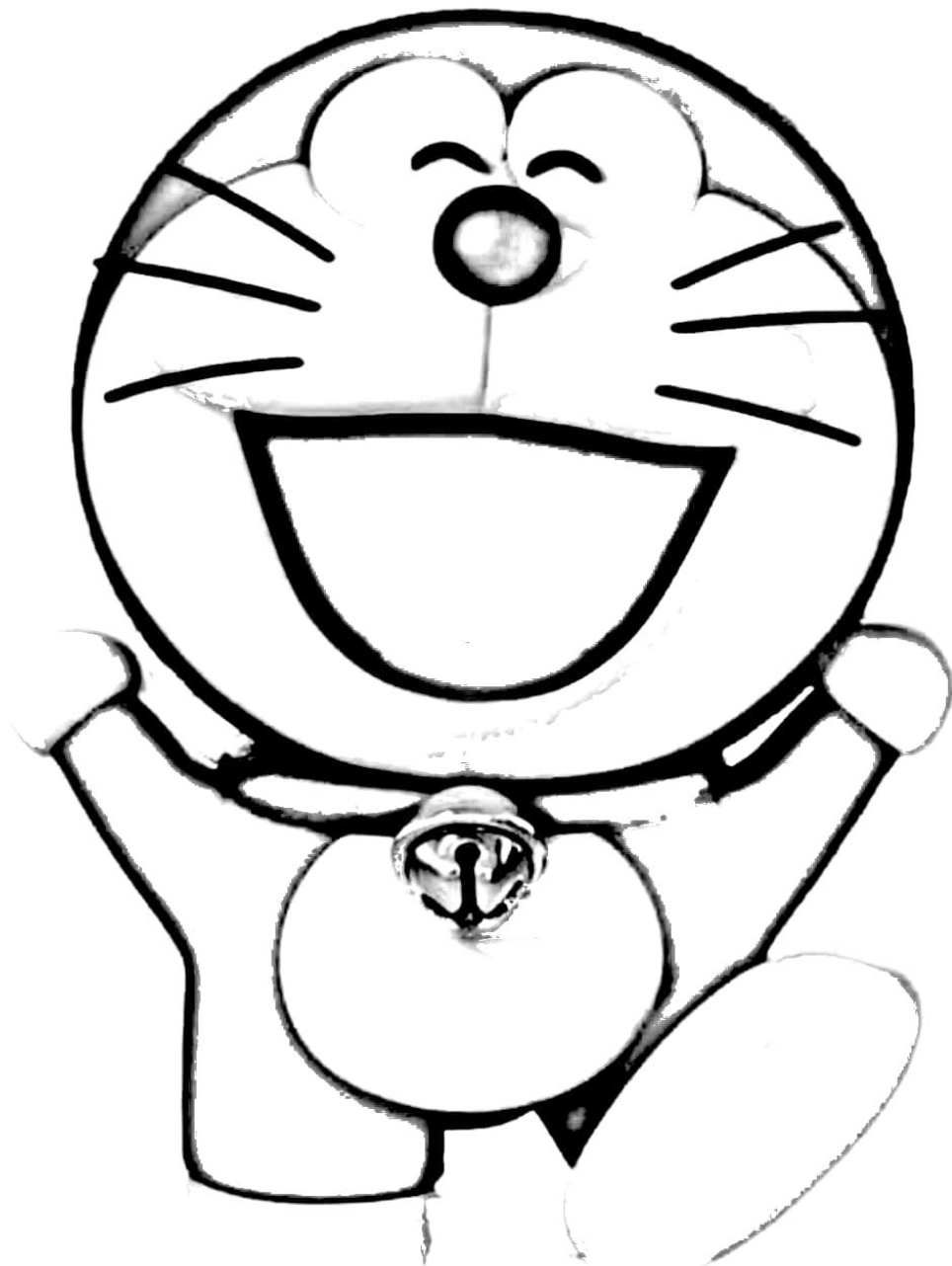
(iv) Array elements are independent to each other.

(iv) Linked list elements are dependent to each other.

(v) Array takes more time.
Ex - insertion, deletion etc.

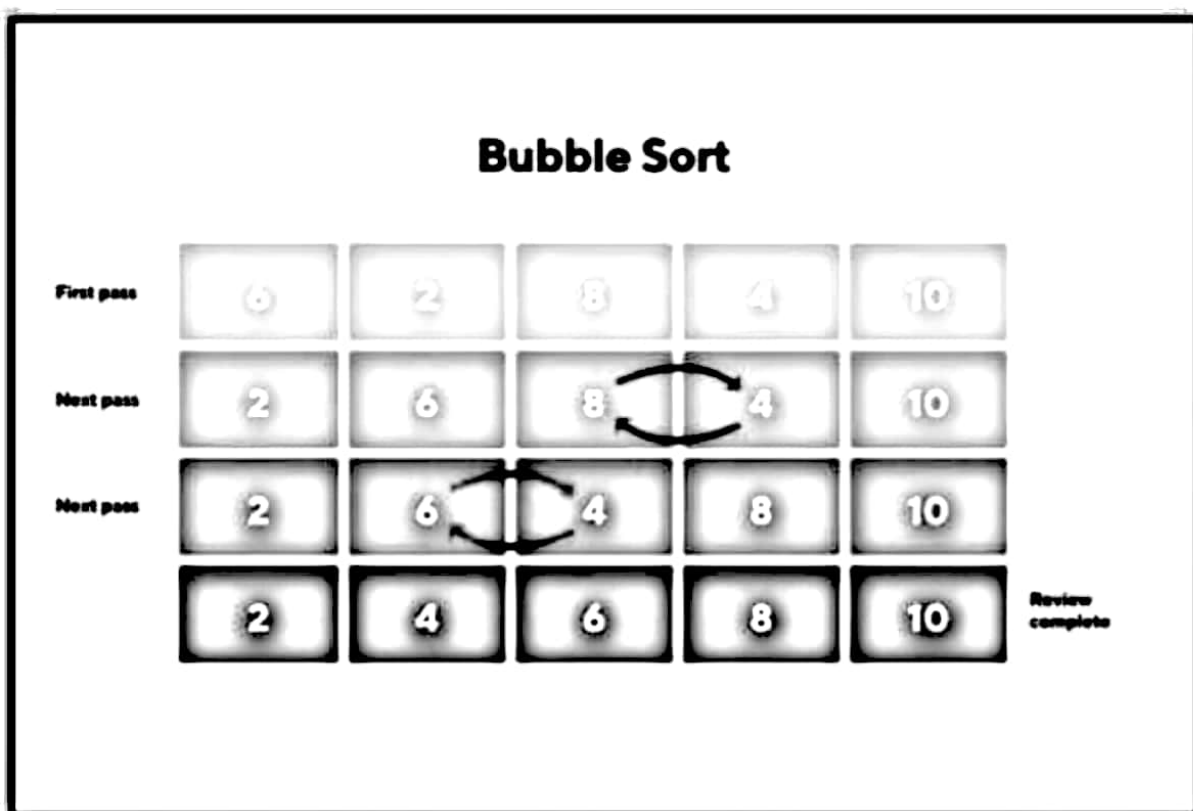
(v) Linked list takes less time.
Ex - Insertion, deletion etc.

Sorting Algorithms



Bubble Sort

Bubble sort is the simplest sorting algorithm that works by repeatedly swapping the adjacent elements if they are in the wrong order. This algorithm is not suitable for large data sets as its average and worst-case time complexities is quite high.



Selection Sort

The Selection sort algorithm sorts an array by repeatedly finding the minimum element (considering ascending order) from unsorted part and putting it at the beginning.

Selection Sort						
1st	12	10	16	11	9	7
2nd	7	10	16	11	9	12
3rd	7	9	16	11	10	12
4th	7	9	10	11	16	12
5th	7	9	10	11	16	12
6th	7	9	10	11	12	16

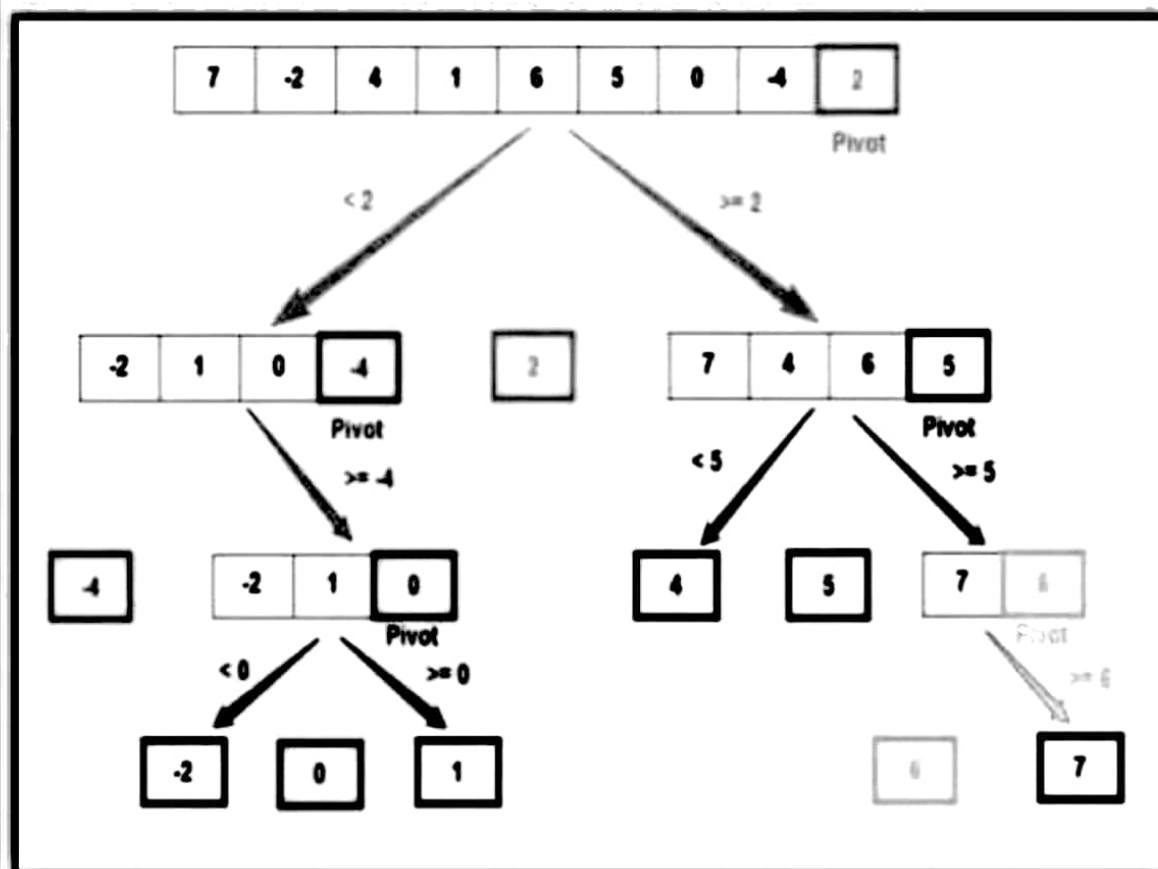
Insertion Sort

Insertion sort is a simple sorting algorithm that works similar to the way you sort playing cards in your hands. The array is virtually split into a sorted and an unsorted part. Values from the unsorted part are picked and placed at the correct position in the sorted part.

54	26	93	17	77	31	44	55	20	Assume 54 is a sorted list of 1 item
26	54	93	17	77	31	44	55	20	inserted 26
26	54	93	17	77	31	44	55	20	inserted 93
17	26	54	93	77	31	44	55	20	inserted 17
17	26	54	77	93	31	44	55	20	inserted 77
17	26	31	54	77	93	44	55	20	inserted 31
17	26	31	44	54	77	93	55	20	inserted 44
17	26	31	44	54	55	77	93	20	inserted 55
17	20	26	31	44	54	55	77	93	inserted 20

Quick Sort

Quick sort is also an divide and conquer algorithm. It picks an element as a pivot and partitions the given array around the picked pivot. This algorithm is used to quickly sort items within an array no matter how big the array is.

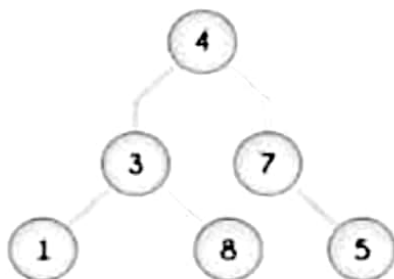


Heap Sort

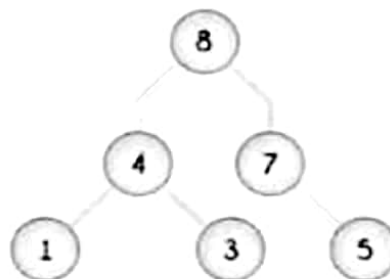
Heap sort is a comparison-based sorting technique based on Binary Heap data structure. It is similar to the selection sort where we first find the minimum element and place the minimum element at the beginning. Repeat the same process for remaining elements.

Arr		4	3	7	1	8	5
	0	1	2	3	4	5	6

Initial Elements

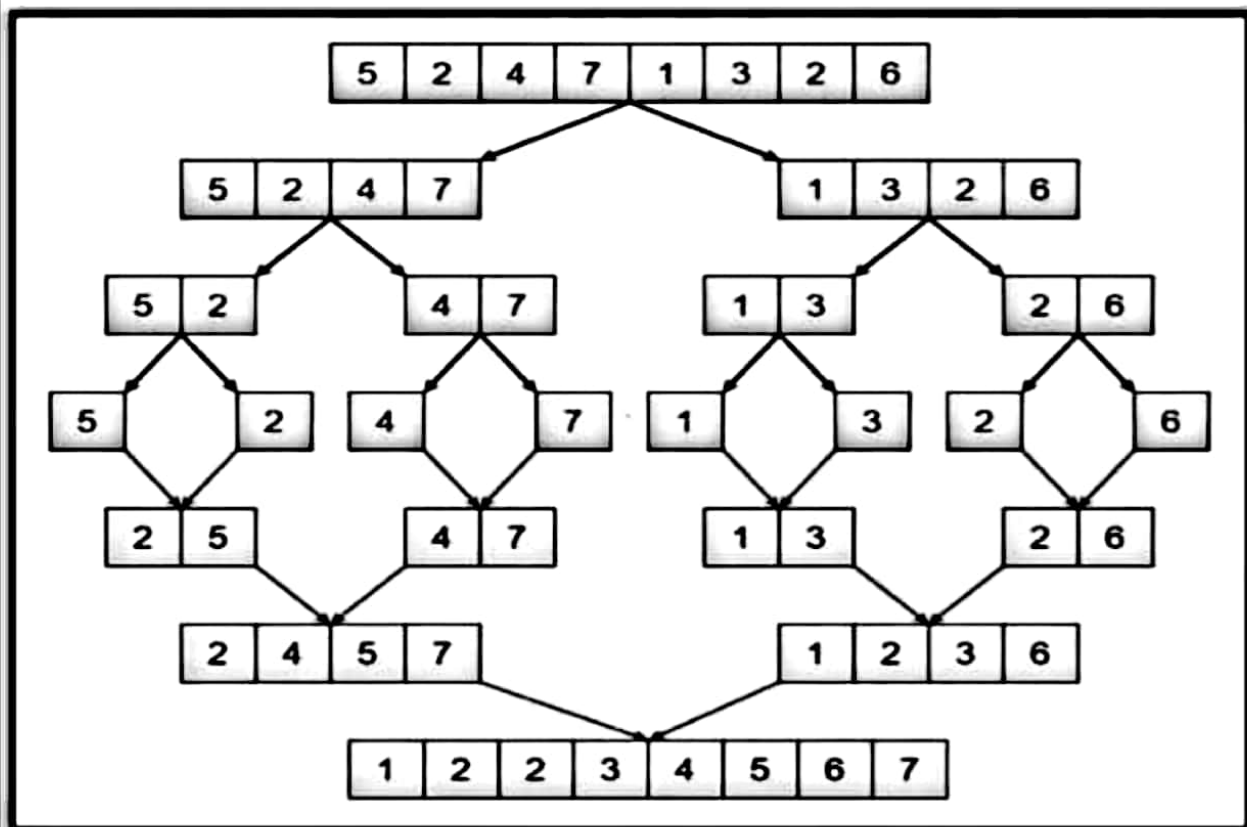


Max Heap



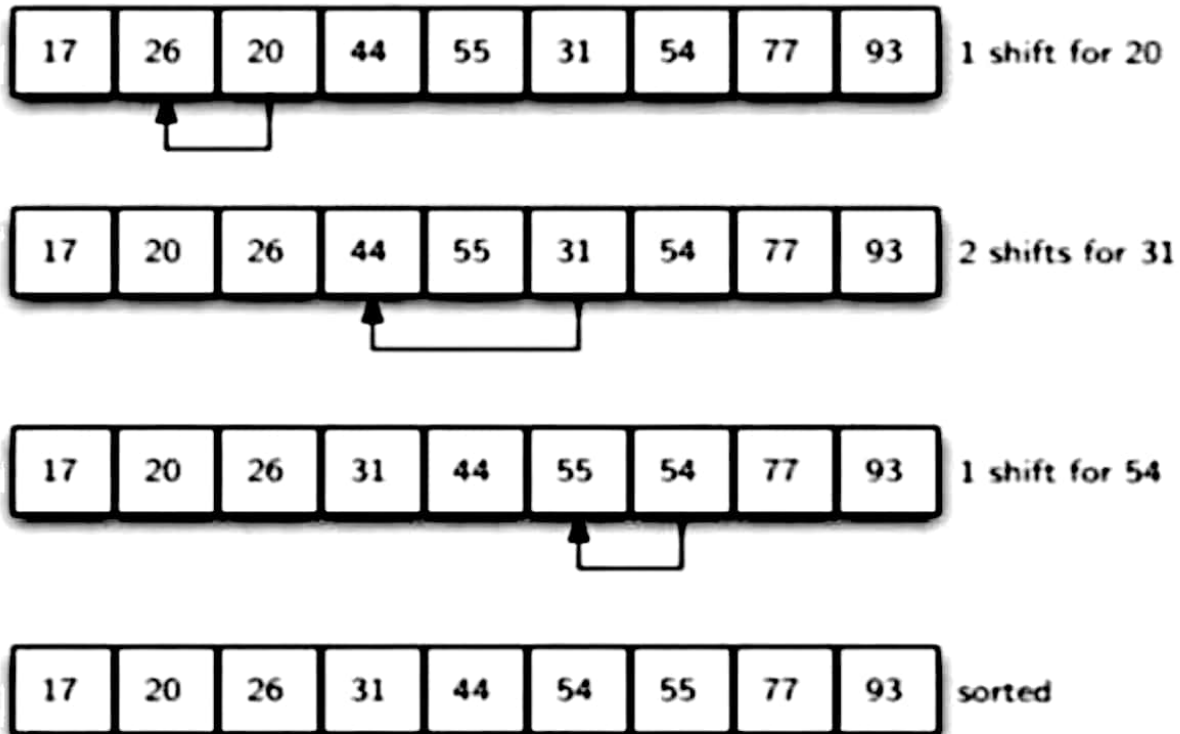
Merge Sort

Merge Sort is a algorithm that is considered an example of the divide and conquer strategy. So, in this algorithm the array is initially divided into two equal halves and then they are combine in a sorted manner.



Shell Sort

Shell sort is mainly a variation of Insertion Sort. In insertion sort, we move elements only one position ahead. When an element has to be moved far ahead, many movements are involved. The idea of ShellSort is to allow the exchange of far items. In Shell sort, we make the array h-sorted for a large value of h. We keep reducing the value of h until it becomes 1. An array is said to be h-sorted if all sublists of every h'th element are sorted.



Time Complexities

CREATE BY - ATUL KUMAR (LINKEDIN)

🐛 CODING BUGS 📖 NOTES GALLERY

Sorting Algorithm	Best Case	Average Case	Worst Case
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Shell Sort	$O(n(\log n)^2)$	$O(n^{3/2})$	$O(n)$